Quantum Computing
Professor Gopal Pandurangan

Scribe Author: Armaan Amatya

Contents

Week 1: August 24–28, 2026 3

Tuesday, August 25, 2026 3
1 Efficient Algorithms and Finding Prime Numbers 3
2 Random Sampling 3
3 Factoring 4
4 Search 4
5 Probability 5

Week 2: August 31 – September 4, 2026 6

Tuesday, September 1, 2026 6
1 Query Complexity for Search 6
2 Coupon Collector Problem 6
3 Lower Bound for Randomized Algorithms 7
4 Chernoff Bounds 8

Thursday, September 3, 2026 9
1 Markov's Inequality 9
2 Chebyshev's Inequality and the Chernoff Bound 9
3 Search 10
4 Birthday Paradox 10
5 Quantum Birthday Paradox 12

Week 3: September 7–11, 2026 14

Tuesday, September 8, 2026 14
1 Distributed Computing in a Complete Network 14
2 A Randomized Algorithm for Leader Election 16
3 Markov Chains: Random Walks in Undirected Graphs 18
4 The Law of Large Numbers 19

Thursday, September 10, 2026 20
1 Random Walks on Graphs 20
2 Stationary Distribution 22
3 Convergence to the Stationary Distribution 22

Week 4: September 14–18, 2026 23

Tuesday, September 15, 2026 23
1 Qubits and Dirac Notation 23
2 Unitary Matrices 24
3 Quantum Circuits and One-Qubit Gates 25

4 The CNOT Gate 26
5 Tensor Products and Entanglement 27

Thursday, September 17, 2026 28
1 The Product Ax as a Combination of Columns 28
2 Linear Independence and Rank 29
3 Orthogonality 29

Week 5: September 21–25, 2026 31

Thursday, September 24, 2026 31
1 Deutsch's Algorithm 31
2 Phase Flip and the n-Qubit Hadamard Transform 32
3 Deutsch–Jozsa Algorithm 33

Week 6: September 28 – October 2, 2026 35

Tuesday, September 29, 2026 35
1 Bernstein–Vazirani Algorithm 35
2 Simon's Algorithm 36

Thursday, October 1, 2026 39
1 Simon's Problem, Classically 39
2 Simon's Algorithm Revisited 43

Week 7: October 5–9, 2026 46

Tuesday, October 6, 2026 46
1 Grover's Algorithm on Two Qubits 46
2 Grover's Algorithm on n Qubits 47
3 Analysis: A Rotation in Two Dimensions 48

Thursday, October 8, 2026 50
1 A Grover Iteration Is Two Reflections 50
2 The Number of Iterations 52
3 A Random Number of Iterations 53
4 Quantum Amplitude Amplification 54

Week: 1 (August 24–28, 2026)

Lecture Date: Tuesday, August 25, 2026

Lecture Topic: Efficient Algorithms, Random Sampling, Lower Bound for Deterministic Search, Probability Basics

In this first lecture, we see why randomness and quantum computation matter for two basic problems, primality/factoring and search. We introduce random sampling as the basic primitive of randomized algorithms, prove a lower bound of $n-1$ queries for deterministic search, and review the basics of probability that we need for the rest of the course.

1 Efficient Algorithms and Finding Prime Numbers

Consider the problem of deciding whether a given number n is prime. The input is the binary representation of n, so the *input size* is the number of bits $N = \log_2 n$.

Definition 1 ▶ Efficient Algorithm

An algorithm is **efficient** if it runs in time polynomial in the input size, i.e., in time $\mathcal{O}\left(N^k\right)$ for some constant k. For primality testing this means time $\mathcal{O}\left((\log_2 n)^k\right)$.

Trial division by all numbers up to $\sqrt{n}$ takes $\Theta(\sqrt{n}) = \Theta\left(2^{N/2}\right)$ time, which is exponential in the input size. A deterministic polynomial-time algorithm exists (time $\tilde{\mathcal{O}}(N^{12})$, later improved to $\tilde{\mathcal{O}}(N^6)$), but it is not used in practice.

1.1 Randomized Algorithms

The algorithm used in practice is *randomized*. The Miller–Rabin test decides primality with high probability in $\mathcal{O}(N^3)$ time. If n is composite, then at least $3/4$ of the numbers in $\{2, \ldots, n-2\}$ are *witnesses* of the fact that n is composite, while a prime has no witness. So after k independent samples a composite n goes undetected with probability at most $(1/4)^k$: the answer "composite" is always right, and the answer "prime" is wrong with probability at most $(1/4)^k$.

2 Random Sampling

All that a randomized algorithm needs is the ability to *sample*. We assume that flipping a (fair) coin is an $\mathcal{O}(1)$ time primitive.

2.1 Sampling a Random Element of an Array

Consider an array with n slots (Figure 1), n a power of 2. Every index is a bit string of $\log_2 n = \log n$ bits, so flipping one coin for each bit samples a uniformly random element in $\mathcal{O}(\log n)$ time: $\mathbb{P}(\text{index } i \text{ is chosen}) = (1/2)^{\log n} = 1/n$ for every i.

We usually assume that an element can be sampled in constant time. This is only justified when the space we sample from has polynomial size in n, since then the number of coin flips is $\mathcal{O}(\log n)$.

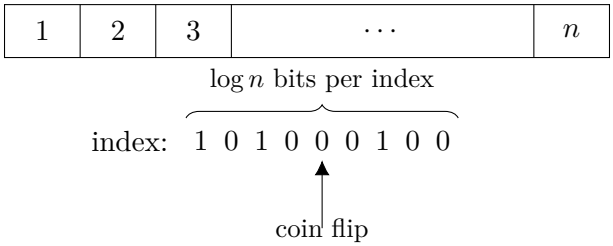


Figure 1: Each of the $\log n$ bits of the index is one coin flip.

2.2 With and Without Replacement

Random sampling can be done *with* or *without* replacement. We almost always analyze sampling with replacement, since without replacement the samples are no longer independent.

Suppose that we are looking for one particular element of the array. Every sample hits it with probability $1/n$, so sampling (with replacement) until it is found takes $\mathbb{E}[\text{samples}] = \frac{1}{1/n} = n$ samples in expectation, and since the samples are independent,

$$\mathbb{P}(\text{not found after } k \text{ samples}) = \left(1 - \frac{1}{n}\right)^k, \qquad \mathbb{P}(\text{found within } k \text{ samples}) = 1 - \left(1 - \frac{1}{n}\right)^k.$$

3 Factoring

Given $n = pq$ with p, q prime, find p and q. Factoring is an *encryption primitive*: if it can be solved quickly, then encryption based on it is broken. No polynomial-time (in $N = \log n$) classical algorithm is known, but *Shor's factoring algorithm* is a quantum algorithm that runs in polynomial time $\mathcal{O}(N^3)$. There is *no* known quantum algorithm that solves NP-complete problems in polynomial time.

Recall that NP is the class of problems whose solutions can be verified in polynomial time, and that the NP-complete problems are the NP problems to which every NP problem reduces (equivalently, the problems that are reducible to 3-SAT and vice versa). Whether P = NP is open. A quantum computer can simulate a classical one with polynomial overhead, so if quantum computers cannot solve NP-complete problems in polynomial time, then P $\neq$ NP.

4 Search

We are given an (unsorted) array A with n slots, which can only be accessed through *queries* ("what is $A[i]$?"), and we want to find the index of a given element x. A sorted array allows binary search with $\mathcal{O}(\log n)$ queries. Linear search makes $\mathcal{O}(n)$ queries.

4.1 Lower Bound for Deterministic Algorithms

An upper bound needs one algorithm, but a lower bound of $\Omega(f(n))$ has to hold for *all* algorithms. We use an *adversary argument*: given any algorithm, the adversary constructs an input on which the algorithm performs badly.

Theorem 1 ▶ Lower Bound for Deterministic Search

Every deterministic algorithm for search makes at least $n - 1$ queries in the worst case.

Proof. Fix a deterministic algorithm. The adversary does not fix the input in advance: whenever the algorithm queries a slot i, it answers that $A[i] \neq x$ (and fills $A[i]$ with some other value).

Since the algorithm is deterministic, its kth query is determined by the answers to the first $k-1$ queries, so this defines one run of the algorithm. Suppose that it stops after $k \leqslant n-2$ queries and outputs an index j. At least $n-k \geqslant 2$ slots have not been queried, so the adversary can place x in an unqueried slot different from j. The resulting array is consistent with all the answers given, and the algorithm is wrong on it. So a correct algorithm makes at least $n-1$ queries. □

4.2 Randomized and Quantum Algorithms

The adversary argument breaks down for *randomized* algorithms, because the queries are no longer determined by the previous answers. Still, randomized search needs $\Omega(n)$ expected queries, by *Yao's min-max principle* (next lecture). Grover's quantum algorithm solves search with $\mathcal{O}(\sqrt{n})$ queries (later in the course).

To show a *quantum advantage* (also called quantum supremacy) for a problem, we have to show that it needs $\Omega(f(n))$ queries classically but only $o(f(n))$ queries quantumly.

5 Probability

A **sample space** S is a finite set of *simple events*, $|S| = n$. A **probability function** $\mathbb{P}\colon S \to [0,1] \subseteq \mathbb{R}$ assigns a probability to every simple event. An **event** is a subset $A \subseteq S$, with probability $\mathbb{P}(A) = \sum_{a \in A} \mathbb{P}(a)$. The probability function satisfies the *axioms* $\mathbb{P}(A) \geqslant 0$ for every event A, $\mathbb{P}(S) = 1$, and $\mathbb{P}(A_1 \cup A_2 \cup \cdots \cup A_n) = \sum_{i=1}^{n} \mathbb{P}(A_i)$ for pairwise disjoint events $A_1, \ldots, A_n$ (i.e., $A_i \cap A_j = \emptyset$ for all $i \neq j$). A **random variable** is a function $X\colon S \to V$ from the sample space to a set of values V (usually $V \subseteq \mathbb{R}$), and for a value $r \in V$, $\mathbb{P}(X = r) = \sum_{s \in S\,:\,X(s)=r} \mathbb{P}(s)$.

Week: 2 (August 31 – September 4, 2026)

Lecture Date: Tuesday, September 1, 2026

Lecture Topic: Query Complexity of Search, Coupon Collector Problem, Lower Bound for Randomized Algorithms, Chernoff Bounds

In this lecture, we study the query complexity of search for classical randomized algorithms. We analyze random sampling, relate it to the coupon collector problem, state a lower bound for randomized search algorithms using Yao's minmax principle, and introduce Chernoff bounds.

1 Query Complexity for Search

Recall the search problem of the last lecture (Figure 1). An array of n slots can only be accessed through *queries*. A query to a slot x returns the value $f(x)$, and the goal is to find the slot x with $f(x) = 1$. The cost of an algorithm is its *query complexity*, the number of queries that it makes.

Definition 2 ▶ Monte Carlo and Las Vegas Algorithms

A **Monte Carlo** algorithm is a randomized algorithm whose error probability is > 0.
A **Las Vegas** algorithm is a randomized algorithm whose error probability is $= 0$.

Random sampling *with replacement* queries in every step a slot chosen uniformly at random. Every query succeeds with probability $1/n$, so the number of queries until the first success is a *geometric* random variable with parameter $1/n$ and $\mathbb{E}[\text{queries until find}] = n$. Stopped after a fixed number of queries, random sampling is a Monte Carlo algorithm (it can fail to find the slot). Run until the slot is found, it is a Las Vegas algorithm with n expected queries.

2 Coupon Collector Problem

To distinguish 2 binary arrays with random sampling, all the n slots have to be accessed. The number of queries needed by random sampling (with replacement) to access all the n slots is the *coupon collector problem*.

Theorem 2 ▶ Coupon Collector

Let X be the number of queries made by random sampling (with replacement) until all the n slots are accessed. Then

$$\mathbb{E}[X] = nH_n = \Theta(n \log n),$$

where $H_n = \sum_{i=1}^{n} \frac{1}{i}$ is the nth harmonic number.

Proof. Write $X = X_1 + X_2 + \cdots + X_n$, where X_i is the number of queries made to access a new slot after $i-1$ different slots have already been accessed. Such a query accesses a new slot with probability $\frac{n-(i-1)}{n}$, so $X_i \sim G\left(\frac{n-(i-1)}{n}\right)$ is geometric with $\mathbb{E}[X_i] = \frac{n}{n-(i-1)}$. By linearity of expectation,

$$\mathbb{E}[X] = \sum_{i=1}^{n} \mathbb{E}[X_i] = n \sum_{i=1}^{n} \frac{1}{n-i+1} = nH_n,$$

and $H_n = \Theta(\log n)$. □

2.1 High Probability Bound

The solution of the following exercise uses the Chernoff bounds and is given in Section 4.1.

Exercise 1. Show that

$$\mathbb{P}(\text{any slot is accessed} \leqslant c \log n \text{ times}) \leqslant \frac{1}{n^2}$$

for a constant $c > 0$, if random sampling makes enough queries (a constant times $n \log n$).

With high probability $3n \log n$ queries are enough. Here log is the natural logarithm.

Theorem 3

If random sampling makes $3n \log n$ queries, then

$$\mathbb{P}(\text{fail}) \leqslant \frac{1}{n^2}, \qquad \text{i.e.,} \qquad \mathbb{P}(\text{Success}) \geqslant 1 - \frac{1}{n^2},$$

where "fail" means that some slot is not accessed.

Proof. Let B_i be the event that slot i is not accessed. A query misses slot i with probability $1 - \frac{1}{n}$ and the queries are independent, so using $1 - x \leqslant e^{-x}$,

$$\mathbb{P}(B_i) \leqslant \left(1 - \frac{1}{n}\right)^{3n \log n} \leqslant e^{-3 \log n} = \frac{1}{n^3}, \qquad \mathbb{P}\left(\bigcup_i B_i\right) \leqslant \sum_{i=1}^{n} \mathbb{P}(B_i) \leqslant \frac{n}{n^3} = \frac{1}{n^2}$$

by the union bound. □

3 Lower Bound for Randomized Algorithms

Theorem 4

Any Las Vegas randomized algorithm for search needs $\Omega(n)$ expected queries.

The lower bound is shown using Yao's minmax principle.

Yao's Minmax Principle

Let A^{rand} be a randomized algorithm, and let $\mathbb{E}\left[A^{\text{rand}}\right]$ be its expected number of queries on its worst input. Let μ be a probability distribution over the inputs, and let D_μ^{best} be the best deterministic algorithm for inputs that are chosen according to μ. Then

$$\mathbb{E}\left[A^{\text{rand}}\right] \geqslant \max_\mu \mathbb{E}_\mu\left[D_\mu^{\text{best}}\right] \geqslant \Omega(n).$$

The quantity $\mathbb{E}_\mu\left[D_\mu^{\text{best}}\right]$ is called the *distributional complexity*. The first inequality is the principle; the second one holds for search and proves Theorem 4:

Proof of Theorem 4. Let μ choose the slot x with $f(x) = 1$ uniformly at random among the n slots, and let D be a deterministic algorithm that is always correct. As long as all the answers are 0, D queries a fixed sequence of different slots $s_1, s_2, \ldots$ (repeating a query gives no new information). It cannot stop while all the answers are 0 and two slots are not yet queried: either

of them could be the solution, and D would be wrong on one of the two inputs. So if the solution is s_j, then D makes at least $\min(j, n-1)$ queries. Every j has probability $1/n$, hence

$$\mathbb{E}_\mu[D] \geqslant \frac{1}{n}\sum_{j=1}^{n} \min(j, n-1) \geqslant \frac{1}{n}\sum_{j=1}^{n-1} j = \frac{n-1}{2} = \Omega(n). \qquad \square$$

4 Chernoff Bounds

If a coin gives heads with probability $\frac{1}{2}+\varepsilon$ and is tossed n times, then $\mathbb{E}[\text{Heads}] = \frac{n}{2} + n\varepsilon$. Chernoff bounds show that the number of heads is close to its expectation, so that $\mathbb{P}\big(\text{Heads} < \frac{n}{2}\big)$ is very small.

In general, let

$$X = \sum_{i=1}^{n} X_i, \qquad X_i = \begin{cases} 1 & \text{with probability } p_i, \\ 0 & \text{with probability } 1-p_i, \end{cases}$$

where the X_i are independent Bernoulli random variables. If $p_i = p$ for all i, then $\mathbb{E}[X] = np$ and $\mathbb{V}[X] = np(1-p)$, and the distribution of X is concentrated around its mean np.

Theorem 5 ▶ Chernoff Bounds

Let $X = \sum_{i=1}^{n} X_i$ be as above and let $\mu = \mathbb{E}[X]$.

- Upper tail (UI): for $0 < \delta < 1$,

$$\mathbb{P}(X \geqslant (1+\delta)\mu) \leqslant e^{-\frac{\mu\delta^2}{3}}.$$

- For $R \geqslant 6\mu$,

$$\mathbb{P}(X \geqslant R) \leqslant \frac{1}{2^R}.$$

- Lower tail (LI): for $0 < \delta < 1$,

$$\mathbb{P}(X < (1-\delta)\mu) \leqslant e^{-\frac{\mu\delta^2}{2}}.$$

4.1 Solution of the Exercise

We solve Exercise 1. As before, log is the natural logarithm. Fix a slot i, suppose that random sampling makes T queries, and let $Y_i = \sum_{t=1}^{T} Z_t$ be the number of times that slot i is accessed, where $Z_t = 1$ if the tth query accesses slot i and $Z_t = 0$ otherwise. The queries are independent (sampling with replacement) and each accesses slot i with probability $1/n$, so the Z_t are independent Bernoulli random variables with $p_t = 1/n$, as in Theorem 5, and $\mu = \mathbb{E}[Y_i] = T/n$.

Lemma 6 ▶ Lower Tail: No Slot Is Accessed Too Rarely

If random sampling makes $24n\log n$ queries, then

$$\mathbb{P}(\text{some slot is accessed} < 12\log n \text{ times}) \leqslant \frac{1}{n^2}.$$

In particular, for $n \geqslant 2$, $\mathbb{P}(\text{some slot is accessed} \leqslant 6\log n \text{ times}) \leqslant 1/n^2$, which is Exercise 1 with $c = 6$.

Proof. Here $T = 24n \log n$, so $\mu = 24 \log n$. The lower tail bound of Theorem 5 with $\delta = 1/2$, for which $(1-\delta)\mu = 12 \log n$, gives for every slot i

$$\mathbb{P}(Y_i < 12 \log n) \leqslant e^{-\frac{\mu\delta^2}{2}} = e^{-\frac{24 \log n}{8}} = \frac{1}{n^3},$$

and by the union bound over the n slots some slot has $Y_i < 12 \log n$ with probability at most $n \cdot 1/n^3 = 1/n^2$. For $n \geqslant 2$ we have $\log n > 0$, so $Y_i \leqslant 6 \log n$ implies $Y_i < 12 \log n$: the event of the second statement is contained in the first and has probability at most $1/n^2$ as well. □

The lemma uses $24n \log n$ queries and not the $3n \log n$ of Theorem 3, because with $\mu = 3 \log n$ the lower tail bound is more than $n^{-3/2}$ per slot, which is too weak for the union bound over the n slots. The exercise also holds with "$\geqslant$" in place of "$\leqslant$" and a larger constant ($c = 144$), by the second bound of Theorem 5 with $R = 6\mu$.

Week: 2 (August 31 – September 4, 2026)

Lecture Date: Thursday, September 3, 2026

Lecture Topic: Markov, Chebyshev and Chernoff Inequalities, Search, Birthday Paradox

---

In this lecture, we state and prove Markov's inequality and Chebyshev's inequality, and we state the Chernoff bound. We then look at the query complexity of search for classical and quantum algorithms, and we analyze the birthday paradox.

1 Markov's Inequality

Markov's inequality uses only the expectation $\mathbb{E}[X]$ of the random variable, which is also called the *first moment*.

Theorem 7 ▶ Markov's Inequality

Let $X \geqslant 0$ be a random variable with $\mathbb{E}[X] > 0$. Then, for any $a > 0$,

$$\mathbb{P}(X \geqslant a\,\mathbb{E}[X]) \leqslant \frac{1}{a}.$$

Equivalently, for any $b > 0$,

$$\mathbb{P}(X \geqslant b) \leqslant \frac{\mathbb{E}[X]}{b}.$$

Proof. Let $I = 1$ if $X \geqslant b$ and $I = 0$ otherwise. Then $I \leqslant \frac{X}{b}$: if $X \geqslant b$ then $I = 1 \leqslant \frac{X}{b}$, and if $X < b$ then $I = 0 \leqslant \frac{X}{b}$ because $X \geqslant 0$. Taking expectations on both sides,

$$\mathbb{P}(X \geqslant b) = \mathbb{P}(I = 1) = \mathbb{E}[I] \leqslant \mathbb{E}\left[\frac{X}{b}\right] = \frac{\mathbb{E}[X]}{b}.$$

Taking $b = a\,\mathbb{E}[X] > 0$ gives $\mathbb{P}(X \geqslant a\,\mathbb{E}[X]) \leqslant \frac{1}{a}$. □

2 Chebyshev's Inequality and the Chernoff Bound

Chebyshev's inequality uses the variance of the random variable, which is also called the *second moment*.

Definition 3 ▶ Variance and Standard Deviation

The **variance** of a random variable X is $\mathbb{V}[X] = \mathbb{E}\Big[(X - \mathbb{E}[X])^2\Big]$. The **standard deviation** of X is $\sigma = \sqrt{\mathbb{V}[X]}$.

Theorem 8 ▶ Chebyshev's Inequality

For any random variable X and any $a > 0$,

$$\mathbb{P}(|X - \mathbb{E}[X]| \geqslant a) \leqslant \frac{\mathbb{V}[X]}{a^2} = \frac{\sigma^2}{a^2}.$$

Proof.

$$\mathbb{P}(|X - \mathbb{E}[X]| \geqslant a) = \mathbb{P}\Big((X - \mathbb{E}[X])^2 \geqslant a^2\Big) \leqslant \frac{\mathbb{E}\Big[(X - \mathbb{E}[X])^2\Big]}{a^2} = \frac{\mathbb{V}[X]}{a^2},$$

where the inequality is Markov's inequality applied to the random variable $(X - \mathbb{E}[X])^2 \geqslant 0$. □

Chernoff bound. The Chernoff bounds of the last lecture (Theorem 5) need $X = \sum_{i=1}^{n} X_i$ to be a sum of *mutually independent* indicator random variables. Then $\mu = \mathbb{E}[X] = \sum_{i=1}^{n} p_i$ with $p_i = \mathbb{P}(X_i = 1)$ by linearity of expectation.

3 Search

Let $f\colon D \to R$ be a function from a domain D to a range R. Given y, we ask: is there an x such that $f(x) = y$? We consider $f\colon [1, \ldots, N] \to [1, \ldots, N]$ and want to find an x such that $f(x) = k$. As in the earlier lectures, the function can be seen as an array with n slots (Figure 1), where n is the size of the domain, and the cost of an algorithm is the number of queries to f. A randomized classical algorithm needs $\Theta(n)$ queries, and Grover's algorithm needs $\mathcal{O}(\sqrt{n})$ queries. For the random sampling algorithm, $\mathbb{E}[\text{queries}] = \mathcal{O}(n)$ and one sample has $\mathbb{P}(\text{Success}) = 1/n$.

Probability amplitude amplification. If the classical success probability is p, then in the quantum case it becomes $\sqrt{p}$.

4 Birthday Paradox

There are m people and n birthdays ($n = 365$), and $f(x)$ is the birthday of person x. We are looking for a *collision*, i.e., $x \neq y$ such that $f(x) = f(y)$.

Let E_i be the event that the ith birthday is different from the first $i - 1$ birthdays. By the chain rule and $1 - x \leqslant e^{-x}$,

$$\mathbb{P}(\text{all birthdays different}) = \mathbb{P}(E_1) \cdot \mathbb{P}(E_2 \mid E_1) \cdot \mathbb{P}(E_3 \mid E_2 \cap E_1) \cdots = \prod_{i=0}^{m-1} \left(1 - \frac{i}{n}\right)$$
$$\leqslant \prod_{i=0}^{m-1} e^{-i/n} = e^{-\frac{(m-1)m}{2n}}.$$

For $m = \sqrt{2n} + 1$ we have $(m-1)m \geqslant 2n$, so this is at most $1/e \leqslant 1/2$. Hence $\mathcal{O}(\sqrt{n})$ people are enough to have a constant probability of collision.

4.1 Expected Number of Collisions

The birthday paradox can be seen as throwing m balls into n bins. We show that for $m \geqslant \sqrt{2n} + 1$, $\mathbb{E}[\#\text{ collisions}] \geqslant 1$.

Let X be the number of collisions. X takes values from 0 to $\binom{m}{2}$. Write

$$X = \sum_{i \neq j} X_{ij}, \qquad X_{ij} = \begin{cases} 1 & \text{if } f(i) = f(j), \\ 0 & \text{else,} \end{cases}$$

where the sum is over the $\binom{m}{2}$ pairs $i \neq j$. The random variables X_{ij} are *pairwise independent*. By linearity of expectation,

$$\mathbb{E}[X] = \sum_{i \neq j} \mathbb{P}(X_{ij} = 1) = \binom{m}{2} \frac{1}{n} = \frac{m(m-1)}{2n} \sim \frac{m^2}{2n} = \Theta\left(\frac{m^2}{n}\right).$$

Exercise 2. Show that

$$\mathbb{P}(X \geqslant 1) \geqslant \Theta(1),$$

i.e., the birthday paradox occurs.

We compute $\mathbb{V}[X]$ and use Chebyshev's inequality (Theorem 8).

Lemma 9 ▶ Variance of the Number of Collisions

For any two different pairs $\{i, j\} \neq \{k, l\}$,

$$\mathbb{E}[X_{ij} X_{kl}] = \mathbb{E}[X_{ij}]\,\mathbb{E}[X_{kl}] = \frac{1}{n^2},$$

i.e., X_{ij} and X_{kl} are independent. Consequently

$$\mathbb{V}[X] = \sum_{i \neq j} \mathbb{V}[X_{ij}] = \binom{m}{2} \frac{1}{n}\left(1 - \frac{1}{n}\right) \leqslant \mathbb{E}[X].$$

Proof. We have $\mathbb{E}[X_{ij}X_{kl}] = \mathbb{P}(f(i) = f(j) \text{ and } f(k) = f(l))$. If the pairs are disjoint, the two events depend on the birthdays of four different people, which are independent, so the probability is $\mathbb{P}(f(i) = f(j)) \cdot \mathbb{P}(f(k) = f(l)) = 1/n^2$. Otherwise the pairs share one person, say $\{i, j\}$ and $\{i, k\}$ with $j \neq k$. Then $f(j)$ and $f(k)$ are independent of $f(i)$ and of each other, and each equals a given day b with probability $1/n$, so

$$\mathbb{P}(f(j) = f(i) \text{ and } f(k) = f(i)) = \sum_{b=1}^{n} \mathbb{P}(f(i) = b) \cdot \frac{1}{n} \cdot \frac{1}{n} = \frac{1}{n^2}.$$

In both cases $\mathbb{E}[X_{ij}X_{kl}] = 1/n^2 = \mathbb{E}[X_{ij}]\,\mathbb{E}[X_{kl}]$. For two indicators this is the same as independence, since their joint distribution is determined by $\mathbb{P}(X_{ij} = 1)$, $\mathbb{P}(X_{kl} = 1)$ and $\mathbb{P}(X_{ij} = X_{kl} = 1)$.

For the variance, expand the square of $X - \mathbb{E}[X] = \sum_{i \neq j}(X_{ij} - \mathbb{E}[X_{ij}])$:

$$\begin{aligned} \mathbb{V}[X] &= \sum_{i \neq j} \mathbb{E}\Big[(X_{ij} - \mathbb{E}[X_{ij}])^2\Big] + \sum_{\{i,j\} \neq \{k,l\}} \mathbb{E}[(X_{ij} - \mathbb{E}[X_{ij}])(X_{kl} - \mathbb{E}[X_{kl}])] \\ &= \sum_{i \neq j} \mathbb{V}[X_{ij}] + \sum_{\{i,j\} \neq \{k,l\}} (\mathbb{E}[X_{ij}X_{kl}] - \mathbb{E}[X_{ij}]\,\mathbb{E}[X_{kl}]) = \sum_{i \neq j} \mathbb{V}[X_{ij}], \end{aligned}$$

since the second sum vanishes by the first part. As $X_{ij}^2 = X_{ij}$, $\mathbb{V}[X_{ij}] = \mathbb{E}\Big[X_{ij}^2\Big] - \mathbb{E}[X_{ij}]^2 = \frac{1}{n}\Big(1 - \frac{1}{n}\Big) \leqslant \frac{1}{n}$, and summing over the $\binom{m}{2}$ pairs gives $\mathbb{V}[X] \leqslant \binom{m}{2}\frac{1}{n} = \mathbb{E}[X]$. □

Theorem 10 ▶ The Birthday Paradox Occurs

If $m \geqslant 2\sqrt{n}+1$, then

$$\mathbb{P}(X \geqslant 1) \geqslant \frac{1}{2}.$$

In words: with $\mathcal{O}(\sqrt{n})$ people there is a collision with constant probability.

Proof. If $X = 0$ then $|X - \mathbb{E}[X]| = \mathbb{E}[X]$. So by Chebyshev's inequality (Theorem 8) with $a = \mathbb{E}[X]$, and then Lemma 9,

$$\mathbb{P}(X = 0) \leqslant \mathbb{P}(|X - \mathbb{E}[X]| \geqslant \mathbb{E}[X]) \leqslant \frac{\mathbb{V}[X]}{\mathbb{E}[X]^2} \leqslant \frac{1}{\mathbb{E}[X]}.$$

For $m \geqslant 2\sqrt{n}+1$, since $m(m-1)$ grows with m, $\mathbb{E}[X] = \frac{m(m-1)}{2n} \geqslant \frac{(2\sqrt{n}+1)\cdot 2\sqrt{n}}{2n} \geqslant 2$, so $\mathbb{P}(X = 0) \leqslant 1/2$ and $\mathbb{P}(X \geqslant 1) = 1 - \mathbb{P}(X = 0) \geqslant 1/2$. □

5 Quantum Birthday Paradox

The birthday paradox solves the *collision problem*: given query access to a function f, find $x \neq y$ with $f(x) = f(y)$. Querying $m = \sqrt{2n}+1$ random points finds a collision with probability at least $1/2$, so $\mathcal{O}(\sqrt{n})$ queries suffice classically. Here we show that classically $\sqrt{n}$ queries are also necessary. An algorithm makes k queries $f(x_1), \ldots, f(x_k)$, where the point x_s may depend on the earlier answers $f(x_1), \ldots, f(x_{s-1})$ and on random coins.

Exercise 3. Show a $\sqrt{n}$ lower bound in the classical case.

As in the birthday paradox, $f\colon D \to [n]$, where $[n] = \{1, \ldots, n\}$, is a *random function*, i.e., every value $f(x)$ is uniform in $[n]$ and independent of all the other values. The algorithm *finds a collision* if it outputs $x \neq y$ with $f(x) = f(y)$. We may assume that it queries the two points that it outputs (at most 2 extra queries, which does not affect the bound $\Omega(\sqrt{n})$) and that the queried points are different, since repeating a query gives no new information.

Theorem 11 ▶ Classical Lower Bound for Collision Finding

Let $f\colon D \to [n]$ be a random function. Every classical algorithm (deterministic or randomized, adaptive) that makes k queries to different points, including the two points that it outputs, finds a collision with probability at most

$$\frac{\binom{k}{2}}{n} = \frac{k(k-1)}{2n}.$$

In particular, to find a collision with probability at least $1/2$ it needs $k > \sqrt{n}$ queries, so the classical query complexity of the collision problem is $\Omega(\sqrt{n})$.

Proof. Since the algorithm queries the points that it outputs, it can only find a collision if two of its queried values agree, i.e., if for some s the event E_s that $f(x_s) \in \{f(x_1), \ldots, f(x_{s-1})\}$ occurs. Condition on the first $s-1$ answers and on the random coins of the algorithm. Then x_s is a fixed point that has not been queried before, so $f(x_s)$ is still uniform in $[n]$ and independent of the $s-1$ values $f(x_1), \ldots, f(x_{s-1})$, which are now fixed. Hence it equals one of them with probability at most $\frac{s-1}{n}$, no matter how the algorithm chose x_s. As this holds for every outcome

of the first $s-1$ steps, $\mathbb{P}(E_s) \leqslant \frac{s-1}{n}$ without conditioning, and by the union bound

$$\mathbb{P}(\text{collision}) \leqslant \sum_{s=1}^{k} \mathbb{P}(E_s) \leqslant \sum_{s=1}^{k} \frac{s-1}{n} = \frac{k(k-1)}{2n}.$$

If the algorithm succeeds with probability at least $1/2$, then $\frac{k(k-1)}{2n} \geqslant \frac{1}{2}$, i.e., $k^2 > k(k-1) \geqslant n$, so $k > \sqrt{n}$. □

Week: 3 (September 7–11, 2026)

Lecture Date: Tuesday, September 8, 2026

Lecture Topic: Distributed Computing, Randomized Leader Election in a Complete Network, Random Walks, Law of Large Numbers

In this lecture, we turn to distributed computing. We define the synchronous complete network model, study the leader election problem, and analyze a randomized algorithm that elects a leader in $\mathcal{O}(1)$ rounds with only $\mathcal{O}\left(\sqrt{n}\log^{3/2} n\right)$ messages. We then take a first look at random walks on undirected graphs, and we prove the weak law of large numbers with Chebyshev's inequality.

1 Distributed Computing in a Complete Network

The network is a *complete graph* G on n nodes (Figure 2). Every pair of nodes is joined by a bidirectional communication link, so there are $\binom{n}{2} = \frac{n(n-1)}{2}$ edges and every node has $n-1$ incident edges. The network is *synchronous*: all nodes share a clock, and the computation proceeds in *rounds*. In one round, every node receives the messages sent to it in the previous round, does a local computation, and sends messages on some of its edges (receives $\to$ computes $\to$ sends).

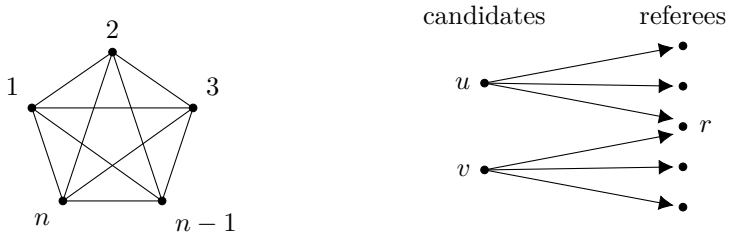


Figure 2: Left: the complete network on n nodes. Right: candidates u and v with a common referee r.

1.1 Leader Election

Problem 1 ▶ Leader Election

All nodes run the same algorithm. At the end, exactly one node must be in the state *leader*, and all other nodes must be in the state *non-leader*.

The goals are to minimize the number of rounds and the total number of messages sent. The nodes are either *labelled*, i.e., every node has a unique id, or *anonymous*, i.e., the nodes have no ids and cannot be told apart.

The trivial algorithm. With unique ids, every node sends its id to all other nodes in the first round, and the node with the minimum id becomes the leader. This takes $\mathcal{O}(1)$ rounds and $n(n-1) = \Theta(n^2)$ messages.

Anonymous nodes. A node addresses its edges by $n-1$ *ports*, numbered $1, \ldots, n-1$, one per incident edge. It can send a message on a given port and sees on which port a message

arrives, but it does not know which node is at the other end of a port, and the port numbering is chosen by an adversary.

Exercise 4. Prove that there is no deterministic algorithm that elects a leader among anonymous nodes in a complete network, no matter how many rounds the algorithm is allowed to run.

The solution is Theorem 12.

Theorem 12 ▶ No Deterministic Leader Election for Anonymous Nodes

Let $n \geqslant 2$. In an anonymous complete network in which all nodes start in the same state, there is no deterministic algorithm that elects a leader for every port numbering: for every deterministic algorithm there is a port numbering for which, after every number of rounds, the number of leaders is either 0 or n.

Proof. Fix a deterministic algorithm. Identify the nodes with $\mathbb{Z}_n = \{0, 1, \ldots, n-1\}$ (addition modulo n), and let port j of node i lead to node $i + j$, for $j = 1, \ldots, n-1$. This is a valid port numbering, since $i+1, \ldots, i+(n-1)$ are exactly the nodes other than i. The edge between i and $i + j$ is port j at i and port $n - j$ at $i + j$, since $(i + j) + (n - j) = i$ in $\mathbb{Z}_n$, so a message sent on port j arrives on port $n - j$.

We show by induction on t that at the end of round t all nodes are in the same state, and that in round t all nodes send the same message m_j on port j, for every j ("no message" counts as a message). For $t = 0$ all nodes are in the same initial state, and nothing has been sent. If the claim holds for round $t - 1$, then in round t node i receives on port q the message that node $i + q$ sent on its port $n - q$, which is the common message m_{n-q} of round $t - 1$ and does not depend on i. So all nodes are in the same state and receive the same messages on the same ports, and since the algorithm is deterministic, they compute the same new state and send the same messages on the same ports.

Whether a node declares itself the leader is a function of its state. Therefore, at the end of every round, either all $n \geqslant 2$ nodes are leaders or no node is, and there is never exactly one leader. □

From now on we assume that the nodes have unique ids.

Complexity classes. Deterministic, bounded-error probabilistic and bounded-error quantum polynomial time satisfy $\mathrm{P} \subseteq \mathrm{BPP} \subseteq \mathrm{BQP}$.

1.2 Known Bounds

For leader election in a complete network of n nodes with unique ids, the following bounds are known ($\widetilde{O}$ and $\widetilde{\Omega}$ hide factors polynomial in $\log n$). We prove only the first and the third row.

Messages	Rounds	Algorithm
$\Theta(n^2)$	$\mathcal{O}(1)$	trivial, deterministic
$\mathcal{O}(n \log n)$	$\mathcal{O}(\log n)$	deterministic
$\mathcal{O}\left(\sqrt{n} \log^{3/2} n\right) = \widetilde{O}(\sqrt{n})$	$\mathcal{O}(1)$	randomized (this lecture)
$\widetilde{\Omega}(\sqrt{n})$		classical lower bound
$\widetilde{O}(n^{1/3})$		quantum

2 A Randomized Algorithm for Leader Election

In this section, log is the natural logarithm, $p = \frac{2\log n}{n}$ and $k = \lceil \sqrt{n \log n} \rceil$. We have $p \leqslant 1$ and $k \leqslant n$ for every $n \geqslant 1$, since $2\log n \leqslant n$. The algorithm has four steps.

1. **Subsample candidates.** Every node becomes a *candidate* with probability p, independently of all other nodes.
2. **Candidates contact referees.** Every candidate sends its id to k random nodes, its *referees*.
3. **Referee round.** Every node that received at least one id declares the minimum id among the received messages the *winner*, and replies with the winner's id to all candidates that contacted it.
4. **Decision.** A candidate becomes the leader if it is the winner at all of its referees. All other nodes are non-leaders.

The model for the random choices. Every node u draws a coin $C_u \in \{0,1\}$ with $\mathbb{P}(C_u = 1) = p$ (u is a candidate if $C_u = 1$) and a *referee set* R_u, which is a uniformly random set of k distinct nodes among all n nodes. All the C_u and all the R_u are mutually independent. Only candidates use their referee sets. A candidate may pick itself as a referee; then it plays the role of this referee itself, and no message is needed.

2.1 The Number of Candidates

Let $X = \sum_u C_u$ be the number of candidates. By linearity of expectation, $\mathbb{E}[X] = np = 2\log n$.

Exercise 5. Show that, with high probability, there is at least 1 leader.

The solution is Theorem 13.

Theorem 13 ▶ At Least One Leader

There is at least one leader if and only if there is at least one candidate, and

$$\mathbb{P}(\text{no leader}) = \mathbb{P}(X = 0) \leqslant \frac{1}{n^2}, \qquad \text{i.e.,} \qquad \mathbb{P}(\text{at least 1 leader}) \geqslant 1 - \frac{1}{n^2}.$$

Proof. Only candidates become leaders, so if $X = 0$ there is no leader. Conversely, if $X \geqslant 1$, let w be the candidate with the minimum id (unique, since the ids are unique). Every referee r of w receives only ids of candidates, all at least the id of w, and the id of w is one of them, so r declares w the winner. Hence w is a leader. The coins are independent, so, using $0 \leqslant 1 - p$ and $1 - x \leqslant e^{-x}$, $\mathbb{P}(X = 0) = (1-p)^n \leqslant e^{-pn} = e^{-2\log n} = 1/n^2$. □

2.2 At Most One Leader

Exercise 6. Show that, with probability tending to 1, there is at most 1 leader.

The solution is Theorem 16, which uses Lemmas 14 and 15; together with Theorem 13 it gives exactly one leader with probability tending to 1 (Theorem 17). Call a pair of different nodes $\{u, v\}$ a *bad pair* if both are candidates and they have no common referee, i.e., $C_u = C_v = 1$ and $R_u \cap R_v = \emptyset$.

Lemma 14 ▶ Two Leaders Form a Bad Pair

If two candidates have a common referee, then at most one of them is a leader. Consequently, if there is more than one leader, then there is a bad pair.

Proof. Let $u \neq v$ be candidates with a common referee r, where the id of u is smaller than the id of v (Figure 2). The referee r receives both ids (if r is u or v itself, it knows its own id), so the minimum id that it receives is at most the id of u. Hence r does not declare v the winner, and v is not a leader. So two different leaders are two candidates without a common referee, i.e., a bad pair. □

Lemma 15 ▶ Two Referee Sets Rarely Miss Each Other

For any two different nodes u and v,

$$\mathbb{P}(R_u \cap R_v = \emptyset) \leqslant \left(1 - \frac{k}{n}\right)^k \leqslant e^{-k^2/n} \leqslant \frac{1}{n}.$$

Proof. Condition on $R_u = S$, where S is any set of k nodes; by independence, R_v is still a uniformly random set of k nodes. If $2k > n$, two sets of k nodes cannot be disjoint and the probability is 0. So let $2k \leqslant n$, and draw the k nodes of R_v one after the other, each uniformly among the nodes not drawn yet. If the first i nodes drawn are outside S, then $n - i$ nodes are left and k of them are in S, so the next node is outside S with probability $\frac{n-k-i}{n-i}$. Hence

$$\mathbb{P}(R_v \cap S = \emptyset) = \prod_{i=0}^{k-1}\left(1 - \frac{k}{n-i}\right) \leqslant \left(1 - \frac{k}{n}\right)^k \leqslant e^{-k^2/n} \leqslant e^{-\log n} = \frac{1}{n},$$

because every factor satisfies $0 \leqslant 1 - \frac{k}{n-i} \leqslant 1 - \frac{k}{n}$ (as $i \leqslant k-1$ and $2k \leqslant n$ give $n - i \geqslant k$), and by $1 - x \leqslant e^{-x}$ and $k^2 \geqslant n \log n$. The bound does not depend on S, so it also holds without conditioning. □

Theorem 16 ▶ At Most One Leader

$$\mathbb{P}(\text{more than 1 leader}) \leqslant \frac{2\log^2 n}{n}.$$

Proof. Let B be the number of bad pairs. For a fixed pair $\{u, v\}$, the coins C_u, C_v and the pair of sets (R_u, R_v) are independent, so by Lemma 15 the pair is bad with probability $p^2 \cdot \mathbb{P}(R_u \cap R_v = \emptyset) \leqslant p^2/n$. By linearity of expectation over the $\binom{n}{2}$ pairs, $\mathbb{E}[B] \leqslant \binom{n}{2}\frac{p^2}{n} = \frac{n(n-1)}{2} \cdot \frac{4\log^2 n}{n^3} \leqslant \frac{2\log^2 n}{n}$. If there is more than one leader, then $B \geqslant 1$ by Lemma 14. Since $B \geqslant 0$, Markov's inequality (Theorem 7) with $b = 1$ gives $\mathbb{P}(\text{more than 1 leader}) \leqslant \mathbb{P}(B \geqslant 1) \leqslant \mathbb{E}[B]$. □

Theorem 17 ▶ Exactly One Leader

The algorithm elects exactly one leader with probability at least

$$1 - \frac{1}{n^2} - \frac{2\log^2 n}{n}.$$

Proof. The algorithm fails if there is no leader or more than one leader. By the union bound and Theorems 13 and 16, this has probability at most $\frac{1}{n^2} + \frac{2\log^2 n}{n}$. □

2.3 Messages and Rounds

Theorem 18 ▶ Complexity of the Algorithm

The algorithm takes 3 rounds, and messages are sent only in the first two. The expected number of messages is at most $4k \log n = \mathcal{O}\left(\sqrt{n} \log^{3/2} n\right)$.

Proof. Rounds. In round 1 every node draws its coin and its referee set, and the candidates send their ids. In round 2 the referees receive the ids and reply with the minimum. In round 3 the candidates receive the replies and decide; no message is sent.

Messages. Every candidate sends at most k messages and gets at most k replies, so the number of messages is at most $2kX$, with expectation at most $2k\,\mathbb{E}[X] = 4k \log n$, which is $\mathcal{O}\left(\sqrt{n} \log^{3/2} n\right)$ since $k \leqslant \sqrt{n \log n} + 1$. □

3 Markov Chains: Random Walks in Undirected Graphs

Definition 4 ▶ Random Walk

Let G be an undirected graph in which every node has at least one neighbour. A **random walk** on G is the following process: a walker starts at some node, and in every step it moves from its current node u to a neighbouring node chosen with uniform probability, i.e., to each neighbour of u with probability $\frac{1}{\deg(u)}$, independently of the earlier steps.

We describe the walk by the row vector $p_t = (p_t^1, \ldots, p_t^n)$, where p_t^i is the probability that the walker is at node i after t steps. The walker is at one node, so the entries of p_t sum to 1. If the walker is at node j after t steps, it moves to a given neighbour i of j with probability $\frac{1}{\deg(j)}$, so by the law of total probability

$$p_{t+1}^i = \sum_{j\,:\,j \text{ is a neighbour of } i} \frac{p_t^j}{\deg(j)}. \tag{1}$$

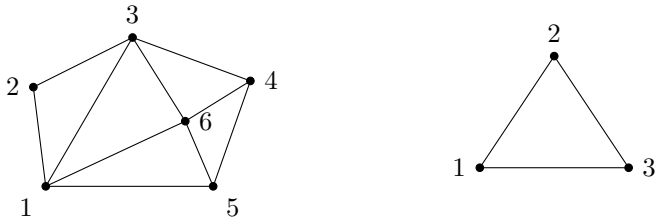


Figure 3: Left: a graph with 6 nodes and 10 edges. Right: the triangle.

Example 1. In the graph on the left of Figure 3, a walker that starts at node 1 has $p_0 = (1,0,0,0,0,0)$ and, since node 1 has the 4 neighbours 2, 3, 5, 6, $p_1 = \left(0, \frac{1}{4}, \frac{1}{4}, 0, \frac{1}{4}, \frac{1}{4}\right)$. In the triangle on the right every node has degree 2, and starting at node 1, by Equation (1), $p_0 = (1,0,0)$, $p_1 = \left(0, \frac{1}{2}, \frac{1}{2}\right)$ and $p_2 = \left(\frac{1}{2}, \frac{1}{4}, \frac{1}{4}\right)$.

Exercise 7. Compute $p_t = (p_t^1, p_t^2, p_t^3)$ for all $t \geqslant 0$ for the random walk on the triangle that starts at node 1.

The solution is Lemma 19.

Lemma 19 ▶ Random Walk on the Triangle

For the random walk on the triangle that starts at node 1, for all $t \geqslant 0$,

$$p_t^1 = \frac{1}{3} + \frac{2}{3}\left(-\frac{1}{2}\right)^t, \qquad p_t^2 = p_t^3 = \frac{1}{3} - \frac{1}{3}\left(-\frac{1}{2}\right)^t.$$

In particular, p_t converges to $\left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right)$ as $t \to \infty$.

Proof. Let i be any of the three nodes, and let j and l be the other two. Both are neighbours of i and have degree 2, so by Equation (1), and since $p_t^i + p_t^j + p_t^l = 1$, $p_{t+1}^i = \frac{1}{2}\left(p_t^j + p_t^l\right) = \frac{1}{2}(1 - p_t^i)$, and hence $p_{t+1}^i - \frac{1}{3} = -\frac{1}{2}\left(p_t^i - \frac{1}{3}\right)$. By induction on t, $p_t^i = \frac{1}{3} + \left(p_0^i - \frac{1}{3}\right)\left(-\frac{1}{2}\right)^t$. With $p_0^1 = 1$ and $p_0^2 = p_0^3 = 0$ this gives the two formulas, and $\left(-\frac{1}{2}\right)^t \to 0$ gives the limit. □

4 The Law of Large Numbers

Exercise 8. Show the "law of large numbers" with Chebyshev's inequality.

The solution is Theorem 20, the *weak* law of large numbers.

Theorem 20 ▶ Weak Law of Large Numbers

Let $X_1, X_2, \ldots$ be pairwise independent random variables with the same expectation $\mathbb{E}[X_i] = \mu$ and the same finite variance $\mathbb{V}[X_i] = \sigma^2$ (for example, independent and identically distributed with finite variance), and let $\overline{X}_n = \frac{1}{n}\sum_{i=1}^n X_i$. Then, for every $\varepsilon > 0$,

$$\mathbb{P}\left(\left|\overline{X}_n - \mu\right| \geqslant \varepsilon\right) \leqslant \frac{\sigma^2}{n\varepsilon^2} \longrightarrow 0 \qquad \text{as } n \to \infty.$$

Proof. By linearity of expectation, $\mathbb{E}\left[\overline{X}_n\right] = \frac{1}{n}\sum_i \mathbb{E}[X_i] = \mu$. Write $Y_i = X_i - \mu$, so that $\mathbb{E}[Y_i] = 0$, $\mathbb{E}[Y_i^2] = \sigma^2$, and $\overline{X}_n - \mu = \frac{1}{n}\sum_i Y_i$. For $i \neq j$, Y_i and Y_j are independent, so $\mathbb{E}[Y_i Y_j] = \mathbb{E}[Y_i]\,\mathbb{E}[Y_j] = 0$, and expanding the square as in the proof of Lemma 9,

$$\mathbb{V}\left[\overline{X}_n\right] = \mathbb{E}\left[\left(\frac{1}{n}\sum_{i=1}^n Y_i\right)^2\right] = \frac{1}{n^2}\sum_{i=1}^n \mathbb{E}\left[Y_i^2\right] + \frac{1}{n^2}\sum_{i \neq j} \mathbb{E}[Y_i Y_j] = \frac{\sigma^2}{n}.$$

Chebyshev's inequality (Theorem 8) for $\overline{X}_n$ with $a = \varepsilon$ gives $\mathbb{P}\left(\left|\overline{X}_n - \mu\right| \geqslant \varepsilon\right) \leqslant \mathbb{V}\left[\overline{X}_n\right]/\varepsilon^2 = \sigma^2/(n\varepsilon^2)$, which tends to 0 as $n \to \infty$. □

Week: 3 (September 7–11, 2026)

Lecture Date: Thursday, September 10, 2026

Lecture Topic: Law of Large Numbers, Random Walks on Graphs, Transition Matrix, Stationary Distribution and Convergence

In this lecture, we state the law of large numbers and return to random walks on graphs. We describe a random walk by its transition matrix P, show that the distribution of the token evolves as $p_{t+1} = p_t P$, find the stationary distribution $\pi = \pi P$, and prove by an analysis of the eigenvalues of P that the walk on a connected, non-bipartite graph converges to it.

Law of large numbers. The average of independent, identically distributed (i.i.d.) samples approaches the expectation.

Theorem 21 ▶ Law of Large Numbers

Let $X_1, X_2, \dots$ be i.i.d. random variables distributed as X, where $\mathbb{E}[X]$ is finite. Then, with probability 1,

$$\lim_{n\to\infty} \frac{\sum_{i=1}^{n} X_i}{n} = \mathbb{E}[X].$$

The weak form of the law was proved in Theorem 20.

1 Random Walks on Graphs

We continue with the random walk of the last lecture on a finite undirected graph $G = (V, E)$ without isolated vertices; the walker is now called the token. We take $V = \{1, \dots, n\}$ and write $m = |E|$ for the number of edges, $u \sim v$ if $\{u, v\} \in E$, $\deg(u)$ (or d_u) for the number of neighbours of u, and X_t for the vertex of the token at time $t = 0, 1, 2, \dots$.

Definition 5 ▶ Distribution at Time t, Transition Matrix

The **distribution at time** t is the row vector

$$p_t = \left(p_t^1, p_t^2, \dots, p_t^n\right), \qquad p_t^i = \mathbb{P}(X_t = i),$$

i.e., p_t^i is the probability that the token is at vertex i at time t (the subscript is the time, the superscript is the vertex). The **transition matrix** is the $n \times n$ matrix P with entries

$$P_{uv} = \mathbb{P}(u \to v) = \mathbb{P}(X_{t+1} = v \mid X_t = u) = \begin{cases} \frac{1}{\deg(u)} & \text{if } u \sim v, \\ 0 & \text{otherwise.} \end{cases}$$

1.1 The Triangle

In the triangle on the vertices $1, 2, 3$ (the right half of Figure 3) every vertex has degree 2, so P is the 3×3 matrix with 0 on the diagonal and $1/2$ in every other entry.

1.2 The Evolution $p_{t+1} = p_t P$

Lemma 22 ▶ Evolution of the Distribution

For every $t \geqslant 0$,

$$p_{t+1} = p_t P, \qquad \text{and hence} \qquad p_t = p_0 P^t.$$

Proof. Fix a vertex v. The events $\{X_t = u\}$, $u \in V$, are disjoint and one of them occurs, so

$$p_{t+1}^v = \mathbb{P}(X_{t+1} = v) = \sum_{u \in V} \mathbb{P}(X_t = u \text{ and } X_{t+1} = v) = \sum_{u \in V} p_t^u P_{uv} = (p_t P)_v,$$

where $\mathbb{P}(X_t = u \text{ and } X_{t+1} = v) = p_t^u P_{uv}$ by the definition of P_{uv} as a conditional probability when $p_t^u > 0$, and both sides are 0 when $p_t^u = 0$. Hence $p_{t+1} = p_t P$, and $p_t = p_0 P^t$ follows by induction on t (with $P^0 = I$). □

Entry by entry, $p_{t+1} = p_t P$ is Equation (1). For the triangle with $p_0 = (1, 0, 0)$, the vectors $p_1 = p_0 P$ and $p_2 = p_0 P^2$ and the closed form of p_t are in Lemma 19 and the example before it.

1.3 Row Stochastic Matrices

Definition 6 ▶ Probability Vector, Row Stochastic Matrix

A vector $p = (p^1, \ldots, p^n)$ is a **probability vector** if $p^u \geqslant 0$ for all u and $\sum_{u=1}^n p^u = 1$. An $n \times n$ matrix P is **row stochastic** if every row of P is a probability vector:

$$P_{uv} \geqslant 0 \text{ for all } u, v, \qquad \text{and} \qquad \sum_{v=1}^{n} P_{uv} = 1 \text{ for all } u.$$

The transition matrix of a random walk on any graph is row stochastic: row u has $\deg(u)$ entries equal to $1/\deg(u)$, and its other entries are 0.

Exercise 9. Show that p_{t+1} is a probability vector, assuming that p_t is.

The solution is Theorem 23.

Theorem 23 ▶ A Row Stochastic Matrix Preserves Probability Vectors

Let P be a row stochastic $n \times n$ matrix and let p be a probability vector. Then pP is a probability vector. Consequently, if p_0 is a probability vector and $p_{t+1} = p_t P$ for all $t \geqslant 0$, then $p_t = p_0 P^t$ is a probability vector for every $t \geqslant 0$.

Proof. Let $q = pP$, so that $q^v = \sum_{u=1}^n p^u P_{uv}$ for every v. Every term is a product of two nonnegative numbers, since p is a probability vector and P is row stochastic, so $q^v \geqslant 0$. Exchanging the two finite sums, and using that every row of P and the entries of p sum to 1, $\sum_v q^v = \sum_v \sum_u p^u P_{uv} = \sum_u p^u (\sum_v P_{uv}) = \sum_u p^u = 1$. Hence pP is a probability vector. By induction on t, every p_t is a probability vector: p_0 is one by assumption, and if p_t is one, then so is $p_t P = p_{t+1}$, which is the statement of Exercise 9. □

1.4 The Transition Matrix of a General Graph

Let A be the *adjacency matrix* of G, i.e., $A_{uv} = 1$ if $u \sim v$ and $A_{uv} = 0$ otherwise, and let D be the diagonal matrix with diagonal entries $d_1, \ldots, d_n$. Its inverse D^{-1}, the diagonal matrix with entries $1/d_1, \ldots, 1/d_n$, exists because there are no isolated vertices. Multiplying A from the left by D^{-1} multiplies row u of A by $1/d_u$, so $(D^{-1}A)_{uv} = A_{uv}/\deg(u)$, which is P_{uv} of

Definition 5, i.e., $P = D^{-1}A$. If G is *d-regular*, i.e., every vertex has the same degree d, then $D = dI$ and $P = A/d$. The triangle is 2-regular, and its P is $A/2$.

2 Stationary Distribution

Definition 7 ▶ Stationary Distribution

A probability vector π is a **stationary distribution** of the walk with transition matrix P if

$$\pi P = \pi \cdot 1 = \pi,$$

i.e., π is a left eigenvector of P with eigenvalue 1.

By Lemma 22, if $p_t = \pi$ then $p_{t+1} = \pi P = \pi$. Only the neighbours v of u have $P_{vu} \neq 0$, and for them $P_{vu} = 1/\deg(v)$, so (as in Equation (1))

$$(\pi P)_u = \sum_{v \in V} \pi_v P_{vu} = \sum_{v \sim u} \frac{\pi_v}{\deg(v)}. \tag{2}$$

Theorem 24 ▶ Stationary Distribution of a Random Walk on a Graph

Let G be a graph with m edges and no isolated vertices. Then

$$\pi_v = \frac{\deg(v)}{2m} \qquad (v \in V)$$

is a stationary distribution of the random walk on G.

Proof. *π is a probability vector.* Since there are no isolated vertices and V is not empty, $m \geqslant 1$, so π is well defined, and $\pi_v \geqslant 0$. Every edge $\{u, v\}$ is counted exactly twice in $\sum_{v \in V} \deg(v)$, once in $\deg(u)$ and once in $\deg(v)$, so $\sum_{v \in V} \deg(v) = 2m$ (the handshake lemma) and $\sum_{v \in V} \pi_v = 1$.

$\pi P = \pi$. For every vertex u, by Equation (2) and $\deg(v) \geqslant 1$, $(\pi P)_u = \sum_{v \sim u} \frac{\deg(v)}{2m} \cdot \frac{1}{\deg(v)} = \sum_{v \sim u} \frac{1}{2m} = \frac{\deg(u)}{2m} = \pi_u$, since the sum has one term for each of the $\deg(u)$ neighbours of u. □

For a d-regular graph on n vertices, $2m = nd$, so $\pi_v = 1/n$ and the stationary distribution is uniform. For the triangle it is $\pi = (1/3, 1/3, 1/3)$, the limit in Lemma 19.

3 Convergence to the Stationary Distribution

Convergence of vectors is entry by entry. We state the theorem without proof.

Theorem 25 ▶ Convergence of Random Walks

Let G be a connected, non-bipartite graph with m edges. Then for every starting distribution p_0,

$$p_t = p_0 P^t \longrightarrow \pi \qquad \text{as } t \to \infty, \qquad \text{where } \pi_v = \frac{\deg(v)}{2m}.$$

In particular, π is the only stationary distribution of the walk.

Week: 4 (September 14–18, 2026)

Lecture Date: Tuesday, September 15, 2026

Lecture Topic: Qubits and Dirac Notation, Unitary Matrices, Quantum Gates and Circuits, Tensor Products, Entanglement

In this lecture, we introduce the language of quantum computing: qubits and Dirac (bra-ket) notation, unitary matrices, and quantum circuits. We meet the gates X, H and CNOT, define the tensor product that combines several qubits into one state, and show that some two-qubit states are entangled.

1 Qubits and Dirac Notation

A classical *bit* takes one of the two values 0 and 1. A quantum bit can be in a combination of both.

Definition 8 ▶ Qubit, n-Qubit State

The state of one **qubit** is a unit vector in $\mathbb{C}^2$,

$$\begin{pmatrix}\alpha\\ \beta\end{pmatrix} = \alpha\begin{pmatrix}1\\ 0\end{pmatrix} + \beta\begin{pmatrix}0\\ 1\end{pmatrix} = \alpha|0\rangle + \beta|1\rangle, \qquad |\alpha|^2 + |\beta|^2 = 1,$$

where $|0\rangle = \begin{pmatrix}1\\ 0\end{pmatrix}$ and $|1\rangle = \begin{pmatrix}0\\ 1\end{pmatrix}$ form the **computational basis**. The complex numbers α, β are the **amplitudes**. More generally, the state of n qubits is a unit vector in $\mathbb{C}^{2^n}$,

$$|\phi\rangle = \sum_{x\in\{0,1\}^n} \alpha_x|x\rangle, \qquad \sum_{x\in\{0,1\}^n} |\alpha_x|^2 = 1,$$

where $|x\rangle$ is the standard basis vector with a 1 in the position indexed by the bit string x (the 2^n strings are listed in increasing binary order) and 0 elsewhere. Measuring the state gives the outcome x with probability $|\alpha_x|^2$.

For one qubit, a measurement gives 0 with probability $|\alpha|^2$ and 1 with probability $|\beta|^2$, and the normalization says that these probabilities sum to 1.

Two qubits. The state of two qubits has four amplitudes, one for each of $00, 01, 10, 11$: $(\alpha, \beta, \gamma, \delta)^T = \alpha|00\rangle+\beta|01\rangle+\gamma|10\rangle+\delta|11\rangle$ with $|\alpha|^2+|\beta|^2+|\gamma|^2+|\delta|^2 = 1$, where $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ are the four standard basis column vectors of $\mathbb{C}^4$, in this order.

Exercise 10. What is the third entry of the column vector $|00\rangle$?

Solution. It is 0. The four entries of a two-qubit column vector are the amplitudes of $|00\rangle, |01\rangle, |10\rangle, |11\rangle$, in this order, so the third entry is the amplitude of $|10\rangle$. The state $|00\rangle = 1\cdot|00\rangle + 0\cdot|01\rangle + 0\cdot|10\rangle + 0\cdot|11\rangle$ has amplitude 0 on $|10\rangle$, i.e., $|00\rangle = (1,0,0,0)^T$. □

Writing a column vector as a combination of the kets $|x\rangle$ is *Dirac notation*. The state of four qubits is a column vector with $2^4 = 16$ entries, but $\frac{1}{\sqrt{2}}|0000\rangle + \frac{1}{\sqrt{2}}|1111\rangle$ is written with two terms: it has $1/\sqrt{2}$ in the first and in the last position and 0 in the other fourteen.

1.1 Kets, Bras and the Dagger

A *ket* $|\phi\rangle$ is a column vector. The corresponding *bra* is the row vector $\langle\phi| = (|\phi\rangle)^\dagger$, where $\dagger$ ("dagger") is the *conjugate transpose*: transpose the vector or matrix and replace every entry by its complex conjugate. For $|\phi\rangle = \begin{pmatrix}\alpha\\ \beta\end{pmatrix}$ we get $\langle\phi| = \begin{pmatrix}\overline{\alpha} & \overline{\beta}\end{pmatrix}$. The dagger equals the inverse only for unitary matrices (Definition 9).

A bra times a ket is a number, the *inner product*, and it gives the (Euclidean) norm: for $|x\rangle, |y\rangle \in \mathbb{C}^n$ with entries x_i, y_i,

$$\langle x|y\rangle = \langle x||y\rangle = \sum_{i=1}^{n} \overline{x_i}\, y_i, \qquad \||y\rangle\|^2 = \langle y|y\rangle = \sum_{i=1}^{n} |y_i|^2.$$

The inner product is linear in the second argument and conjugate-linear in the first, and $\langle y|x\rangle = \overline{\langle x|y\rangle}$. We use two rules for the dagger: $(A^\dagger)^\dagger = A$, and $(AB)^\dagger = B^\dagger A^\dagger$ for matrices A, B whose product is defined; in particular $(U|x\rangle)^\dagger = \langle x|U^\dagger$.

2 Unitary Matrices

Definition 9 ▶ Unitary Matrix

An $n \times n$ complex matrix U is **unitary** if $U^\dagger U = I$.

Exercise 11. Show that $\|U|x\rangle\|_2 = \||x\rangle\|_2$ for all $|x\rangle \in \mathbb{C}^n$ if and only if U is a unitary matrix ($U^\dagger U = I$).

The solution is Theorem 26.

Theorem 26 ▶ Norm Preserving $\iff$ Unitary

Let U be an $n \times n$ complex matrix. Then $\|U|x\rangle\| = \||x\rangle\|$ for all $|x\rangle \in \mathbb{C}^n$ if and only if $U^\dagger U = I$.

Proof. Unitary implies norm preserving. If $U^\dagger U = I$, then for every $|x\rangle$, $\|U|x\rangle\|^2 = (U|x\rangle)^\dagger U|x\rangle = \langle x|U^\dagger U|x\rangle = \langle x|x\rangle = \||x\rangle\|^2$, and norms are non-negative, so $\|U|x\rangle\| = \||x\rangle\|$.

Norm preserving implies unitary. Assume $\|U|x\rangle\| = \||x\rangle\|$ for all $|x\rangle$, and let $M = U^\dagger U - I$. We show $M = 0$, using two properties of M.

1. M is Hermitian: $M^\dagger = (U^\dagger U)^\dagger - I^\dagger = U^\dagger (U^\dagger)^\dagger - I = M$.
2. For every $|x\rangle$, since $\|U|x\rangle\|^2 = \langle x|U^\dagger U|x\rangle$ for every matrix U, $\langle x|M|x\rangle = \langle x|U^\dagger U|x\rangle - \langle x|x\rangle = \|U|x\rangle\|^2 - \||x\rangle\|^2 = 0$.

Fix $|x\rangle, |y\rangle \in \mathbb{C}^n$ and let $z = \langle x|M|y\rangle$. A number is its own transpose, so its conjugate equals its dagger, and by property 1, $\overline{z} = (\langle x|M|y\rangle)^\dagger = \langle y|M^\dagger|x\rangle = \langle y|M|x\rangle$. Apply property 2 to $|x\rangle + |y\rangle$, whose bra is $\langle x| + \langle y|$, and to $|x\rangle + i|y\rangle$, whose bra is $\langle x| - i\langle y|$ because the dagger conjugates. Expanding, the terms $\langle x|M|x\rangle$ and $\langle y|M|y\rangle$ vanish by property 2, and

$$0 = (\langle x| + \langle y|)M(|x\rangle + |y\rangle) = \langle x|M|y\rangle + \langle y|M|x\rangle = z + \overline{z} = 2\,\mathrm{Re}\, z,$$
$$0 = (\langle x| - i\langle y|)M(|x\rangle + i|y\rangle) = i\langle x|M|y\rangle - i\langle y|M|x\rangle = i(z - \overline{z}) = -2\,\mathrm{Im}\, z.$$

Hence $\langle x|M|y\rangle = z = 0$ for all $|x\rangle, |y\rangle$. For $|x\rangle = e_j$ and $|y\rangle = e_k$, the jth and kth standard basis vectors, Me_k is the kth column of M and $e_j^\dagger$ picks out its jth entry, so $0 = e_j^\dagger M e_k = M_{jk}$ for all j, k. So $M = 0$, i.e., $U^\dagger U = I$. □

Remark. For a square matrix, $U^\dagger U = I$ also gives $UU^\dagger = I$: if $U|x\rangle = 0$ then $|x\rangle = U^\dagger U|x\rangle = 0$, so U is injective, hence invertible, and multiplying $U^\dagger U = I$ by U^{-1} on the right gives $U^\dagger = U^{-1}$, hence $UU^\dagger = I$. So every unitary matrix is invertible, and its inverse $U^\dagger$ is again unitary.

A quantum gate is a linear map that sends every state, i.e., every unit vector, to a state. A linear map that preserves the norm of unit vectors preserves the norm of all vectors (every vector is a multiple of a unit vector), so by Theorem 26 it is unitary. Conversely, every unitary matrix maps states to states.

3 Quantum Circuits and One-Qubit Gates

A *quantum circuit* is drawn with one horizontal wire per qubit. Time runs from left to right, and a box on a wire is a gate, i.e., a unitary matrix applied to that qubit (Figure 4).

Columns of a matrix. Let A be an $m \times n$ matrix with columns $a_1, \ldots, a_n$, and let $x = (x_1, \ldots, x_n)^T$. Then $Ax = x_1a_1 + x_2a_2 + \cdots + x_na_n$ is a linear combination of the columns of A. So $Ae_j = a_j$: *the jth column of a gate's matrix is the image of the jth basis state*, and a gate is determined by what it does to the basis states.

The X gate. The *Pauli X gate* is the quantum NOT gate. It swaps the two basis states, $X|0\rangle = |1\rangle$ and $X|1\rangle = |0\rangle$, so its columns are $|1\rangle$ and $|0\rangle$, and by linearity it swaps the amplitudes of a general state:

$$X = \sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad X(\alpha|0\rangle + \beta|1\rangle) = \alpha|1\rangle + \beta|0\rangle.$$

X is unitary: it is real and symmetric, so $X^\dagger = X$, and $X^\dagger X = X^2 = I$ because swapping twice is the identity.

Definition 10 ▶ Hadamard Gate

The **Hadamard gate** H is the one-qubit gate with

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \qquad H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle), \qquad H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}. \tag{3}$$

The Hadamard gate turns a basis state into an equal superposition: measuring $H|0\rangle$ or $H|1\rangle$ gives 0 and 1 with probability $1/2$ each. The two states differ only in the sign of the amplitude of $|1\rangle$.

Lemma 27 ▶ Hadamard Twice Is the Identity

$H^2 = I$. Consequently H is unitary, and

$$H\,\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |0\rangle, \qquad H\,\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |1\rangle.$$

Proof. By matrix multiplication, the diagonal entries of H^2 are $\frac{1}{2}(1 \cdot 1 + 1 \cdot 1) = 1$ and $\frac{1}{2}(1 \cdot 1 + (-1)(-1)) = 1$, and both off-diagonal entries are $\frac{1}{2}(1 \cdot 1 + 1 \cdot (-1)) = 0$, so $H^2 = I$. H is real and symmetric, so $H^\dagger = H$ and $H^\dagger H = H^2 = I$, i.e., H is unitary. By Equation (3), the two states in the statement are $H|0\rangle$ and $H|1\rangle$, and $HH|0\rangle = |0\rangle$, $HH|1\rangle = |1\rangle$. □

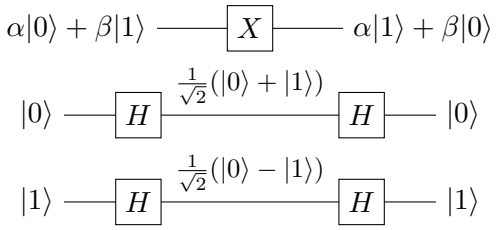


Figure 4: Top: the X gate. Below: the Hadamard gate applied twice (Lemma 27).

4 The CNOT Gate

Definition 11 ▶ CNOT Gate

The **controlled-NOT gate** (CNOT) is the two-qubit gate that maps the basis states by

$$|a\rangle|b\rangle \longmapsto |a\rangle|b \oplus a\rangle, \qquad a, b \in \{0, 1\},$$

where $\oplus$ is XOR (addition modulo 2). The first qubit $|a\rangle$ is the **control** qubit and is left unchanged; the second qubit $|b\rangle$ is the **target** qubit and is flipped exactly when $a = 1$. See Figure 5.

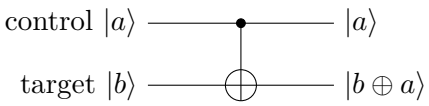


Figure 5: The CNOT gate: a filled dot on the control qubit and $\oplus$ on the target qubit.

The four basis states are mapped by $|00\rangle \mapsto |00\rangle$, $|01\rangle \mapsto |01\rangle$, $|10\rangle \mapsto |11\rangle$ and $|11\rangle \mapsto |10\rangle$ (for example $a = 1$, $b = 1$ gives $b \oplus a = 0$).

Lemma 28 ▶ Matrix of the CNOT Gate

The matrix of the CNOT gate, with rows and columns indexed by $00, 01, 10, 11$, is the permutation matrix

$$U_{\text{CNOT}} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

It is unitary.

Proof. The columns of the matrix of a gate are the images of the basis states. By Definition 11, the images of $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ are $|00\rangle, |01\rangle, |11\rangle, |10\rangle$, i.e., $(1,0,0,0)^T$, $(0,1,0,0)^T$, $(0,0,0,1)^T$, $(0,0,1,0)^T$, which are the four columns of U_{CNOT}, in this order.

The map $\pi\colon (a, b) \mapsto (a, b \oplus a)$ on $\{0,1\}^2$ is its own inverse, since $\pi(\pi(a,b)) = (a, b \oplus a \oplus a) = (a, b)$. Hence π is a permutation of the four basis states, and U_{CNOT} is a *permutation matrix*: every column is a standard basis vector, and different columns are different basis vectors.

Every permutation matrix P is unitary: the (j,k) entry of P^TP is the inner product of the jth and the kth column of P, which are standard basis vectors, equal if $j = k$ and different if $j \neq k$. Thus $P^TP = I$, and since P is real, $P^\dagger P = P^TP = I$. □

In block form, $U_{\text{CNOT}} = \begin{pmatrix} I & 0 \\ 0 & X \end{pmatrix}$: if the control is 0 nothing happens, and if it is 1 the gate X is applied to the target.

4.1 Classical Gates Are Not Reversible

By Theorem 26 and the remark after it, every quantum gate is unitary and therefore invertible: the computation can be undone by applying $U^\dagger$. Classical gates such as AND, OR and XOR, which take two bits to one bit, are *not bijective*, hence *not reversible*, hence *not unitary*.

Lemma 29 ▶ XOR Is Not Reversible

The classical gate $\mathrm{XOR}\colon \{0,1\}^2 \to \{0,1\}$, $(a,b) \mapsto a \oplus b$, is not injective, so its input cannot be recovered from its output. The linear map $T\colon \mathbb{C}^4 \to \mathbb{C}^2$ with $T|ab\rangle = |a \oplus b\rangle$ does not preserve norms, so it is not unitary. The same holds for every gate with two input bits and one output bit.

Proof. $\mathrm{XOR}(0,0) = 0 = \mathrm{XOR}(1,1)$, so two different inputs have the same output, and no function of the output can return the input in both cases. For the linear map, $T(|00\rangle - |11\rangle) = |0\rangle - |0\rangle = 0$, although $\||00\rangle - |11\rangle\| = \sqrt{2} \neq 0$. A general gate $g\colon \{0,1\}^2 \to \{0,1\}$ has four inputs and only two outputs, so by the pigeonhole principle two inputs $u \neq v$ have $g(u) = g(v)$, and the same argument applies with $|u\rangle - |v\rangle$ (for AND, $u = 00$ and $v = 01$; for OR, $u = 01$ and $v = 11$). □

The CNOT gate is the reversible version of XOR: it computes $b \oplus a$ on the target wire and keeps a on the control wire, so from $(a, b \oplus a)$ we recover $b = (b \oplus a) \oplus a$.

5 Tensor Products and Entanglement

Definition 12 ▶ Tensor Product

Let $A = (a_{ij})$ be an $m \times n$ matrix and let B be a $p \times q$ matrix. The **tensor product** (Kronecker product) $A \otimes B$ is the $mp \times nq$ block matrix

$$A \otimes B = \begin{pmatrix} a_{11}B & a_{12}B & \cdots & a_{1n}B \\ a_{21}B & a_{22}B & \cdots & a_{2n}B \\ \vdots & & & \vdots \\ a_{m1}B & a_{m2}B & \cdots & a_{mn}B \end{pmatrix}.$$

Column vectors are matrices with one column, so in particular

$$\begin{pmatrix} \alpha \\ \beta \end{pmatrix} \otimes \begin{pmatrix} \gamma \\ \delta \end{pmatrix} = \begin{pmatrix} \alpha\gamma \\ \alpha\delta \\ \beta\gamma \\ \beta\delta \end{pmatrix}.$$

For basis states this gives, for example, $|1\rangle \otimes |0\rangle = (0,0,1,0)^T = |10\rangle$, and in general $|a\rangle|b\rangle = |a\rangle \otimes |b\rangle = |ab\rangle$ for $a, b \in \{0,1\}$. In terms of kets, $(\alpha|0\rangle + \beta|1\rangle) \otimes (\gamma|0\rangle + \delta|1\rangle) = \alpha\gamma|00\rangle + \alpha\delta|01\rangle + \beta\gamma|10\rangle + \beta\delta|11\rangle$. If A is applied to the first qubit and B to the second, the two-qubit gate is $A \otimes B$, and

$$(A \otimes B)(|x\rangle \otimes |y\rangle) = A|x\rangle \otimes B|y\rangle. \tag{4}$$

If $x_1, \ldots, x_n$ are the entries of $|x\rangle$, then $|x\rangle \otimes |y\rangle$ consists of the blocks $x_1|y\rangle, \ldots, x_n|y\rangle$, so the ith block of the left-hand side is $\sum_j a_{ij} B(x_j|y\rangle) = \left(\sum_j a_{ij} x_j\right) B|y\rangle$, which is the ith block of the right-hand side.

Definition 13 ▶ Product State, Entangled State

A two-qubit state $|\psi\rangle$ is a **product state** if $|\psi\rangle = |\phi_1\rangle \otimes |\phi_2\rangle$ for some one-qubit states $|\phi_1\rangle$, $|\phi_2\rangle$. Otherwise $|\psi\rangle$ is **entangled**.

The two qubits of the *Bell state* $\frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|11\rangle$ are entangled.

Exercise 12. Show that $\frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|11\rangle \neq |\alpha\rangle|\beta\rangle$ for all one-qubit states $|\alpha\rangle$, $|\beta\rangle$.

The solution is Theorem 30.

Theorem 30 ▶ The Bell State Is Entangled

The state $\frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|11\rangle$ is not a product state: there are no vectors $|\phi_1\rangle, |\phi_2\rangle \in \mathbb{C}^2$ with

$$\frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|11\rangle = |\phi_1\rangle \otimes |\phi_2\rangle.$$

Proof. Suppose there are, and write $|\phi_1\rangle = \begin{pmatrix}\alpha\\ \beta\end{pmatrix}$ and $|\phi_2\rangle = \begin{pmatrix}\gamma\\ \delta\end{pmatrix}$. By Definition 12, comparing the four entries of the two sides gives $\alpha\gamma = 1/\sqrt{2}$, $\alpha\delta = 0$, $\beta\gamma = 0$ and $\beta\delta = 1/\sqrt{2}$. From $\alpha\delta = 0$ we get $\alpha = 0$ or $\delta = 0$. If $\alpha = 0$ then $\alpha\gamma = 0 \neq 1/\sqrt{2}$, and if $\delta = 0$ then $\beta\delta = 0 \neq 1/\sqrt{2}$. Both cases are contradictions. □

Week: 4 (September 14–18, 2026)

Lecture Date: Thursday, September 17, 2026

Lecture Topic: Linear Algebra Review: Column Space, Rank, Linear Independence, Orthogonality

---

In this lecture, we review linear algebra: the product Ax as a combination of the columns of A, linear independence, column rank and row rank, and orthogonality.

1 The Product Ax as a Combination of Columns

The product Ax is the linear combination of the columns of A with the entries of x as coefficients. For example, let $A \in \mathbb{R}^{3\times 3}$ be the matrix with columns $v_1 = (2,3,5)^T$, $v_2 = (1,1,7)^T$, $v_3 = (3,4,12)^T$. Then

$$A = \begin{bmatrix} 2 & 1 & 3 \\ 3 & 1 & 4 \\ 5 & 7 & 12 \end{bmatrix}, \qquad Ax = \begin{bmatrix} 2x_1 + x_2 + 3x_3 \\ 3x_1 + x_2 + 4x_3 \\ 5x_1 + 7x_2 + 12x_3 \end{bmatrix} = x_1 v_1 + x_2 v_2 + x_3 v_3. \tag{5}$$

Here $v_3 = v_1 + v_2$, so $Ax = (x_1 + x_3)\, v_1 + (x_2 + x_3)\, v_2$: every output Ax is a combination of v_1 and v_2 alone.

In general, an $m \times n$ matrix A with columns $a_1, \ldots, a_n$ defines the linear map $T\colon \mathbb{R}^n \to \mathbb{R}^m$, $T(x) = Ax = x_1 a_1 + \cdots + x_n a_n$. Its *image*, the set of all outputs, is the *column space* of A:

$$\mathrm{Im}(A) = \{Ax \mid x \in \mathbb{R}^n\} = \mathrm{span}(a_1, \ldots, a_n).$$

2 Linear Independence and Rank

Definition 14 ▶ Linear Independence

Vectors $v_1, \dots, v_k$ are **linearly independent** if

$$c_1 v_1 + c_2 v_2 + \cdots + c_k v_k = 0 \quad \Longrightarrow \quad c_1 = c_2 = \cdots = c_k = 0,$$

i.e., the only combination of them that gives the zero vector is the trivial one. Otherwise they are **linearly dependent**. If $v_1, \dots, v_k$ are linearly independent, we say that a vector v is *independent of* them if $v, v_1, \dots, v_k$ are linearly independent.

Definition 15 ▶ Column Rank and Row Rank

The **column rank** $\operatorname{rank}_c(A)$ of a matrix A is the maximum number of linearly independent columns of A. The **row rank** $\operatorname{rank}_r(A)$ is the maximum number of linearly independent rows of A.

Lemma 31 ▶ Column Rank Is the Dimension of the Image

Let A be an $m \times n$ matrix with $r = \operatorname{rank}_c(A) \geqslant 1$. Then any r linearly independent columns of A form a basis of $\operatorname{Im}(A)$. In particular,

$$\operatorname{rank}_c(A) = \dim(\operatorname{Im}(A)).$$

Exercise 13. For the matrix A of Equation (5), find $\operatorname{rank}_c(A)$, $\operatorname{Im}(A)$ and $\operatorname{rank}_r(A)$, and express every row of A as a combination of [1 0 1] and [0 1 1].

Solution. *Columns.* v_1, v_2 are independent: $av_1 + bv_2 = 0$ gives $2a + b = 0$ and $3a + b = 0$, so $a = 0$ and then $b = 0$. The three columns are dependent, since $v_1 + v_2 - v_3 = 0$. Hence $\operatorname{rank}_c(A) = 2$, and by Lemma 31, $\operatorname{Im}(A) = \operatorname{span}(v_1, v_2)$, of dimension 2.

Rows. Let $u_1 = [1\ 0\ 1]$ and $u_2 = [0\ 1\ 1]$, so that $au_1 + bu_2 = [a \quad b \quad a + b]$. Then

$$[2\ 1\ 3] = 2u_1 + 1u_2, \qquad [3\ 1\ 4] = 3u_1 + 1u_2, \qquad [5\ 7\ 12] = 5u_1 + 7u_2.$$

The three rows are dependent, since $16\,[2\ 1\ 3] - 9\,[3\ 1\ 4] - [5\ 7\ 12] = 0$, so $\operatorname{rank}_r(A) \leqslant 2$. The first two rows are independent: $a\,[2\ 1\ 3] + b\,[3\ 1\ 4] = 0$ gives $a + b = 0$ (second entry) and $2a + 3b = 0$ (first entry), so $a = b = 0$. Hence $\operatorname{rank}_r(A) = 2$. □

Theorem 32 ▶ Row Rank Equals Column Rank

For every $m \times n$ matrix A, $\operatorname{rank}_r(A) = \operatorname{rank}_c(A)$. This common value is called the **rank** of A.

3 Orthogonality

The results of this section hold over $\mathbb{R}$ and over $\mathbb{C}$. The inner product is $\langle a, b\rangle = a \cdot b = \sum_{i=1}^{n} a_i b_i$ for $a, b \in \mathbb{R}^n$, and $\langle a, b\rangle = \sum_{i=1}^{n} \overline{a_i}\, b_i$ (linear in b) for $a, b \in \mathbb{C}^n$. In both cases $\langle a, a\rangle = \sum_{i=1}^{n} |a_i|^2 = \|a\|^2$, which is 0 only for $a = 0$.

Definition 16 ▶ Orthogonality

Two vectors a and b are **orthogonal** if $\langle a, b\rangle = 0$; for real vectors, $a \cdot b = 0$.

Exercise 14. Let $v_1, \dots, v_k$ be linearly independent vectors. Show that if a nonzero vector v is orthogonal to $v_1, v_2, \dots, v_k$, then v is independent of $v_1, v_2, \dots, v_k$.

The solution is Theorem 33.

Theorem 33 ▶ Orthogonal Implies Independent

Let $v_1, \dots, v_k$ be linearly independent vectors in $\mathbb{R}^n$ or $\mathbb{C}^n$, and let $v \neq 0$ be orthogonal to each of them. Then $v, v_1, \dots, v_k$ are linearly independent. In particular, $v \notin \operatorname{span}(v_1, \dots, v_k)$.

Proof. Suppose that $c\,v + c_1 v_1 + \cdots + c_k v_k = 0$. Taking the inner product of v with both sides (linear in the second argument) and using $\langle v, v_i \rangle = 0$,

$$0 = c\,\langle v, v \rangle + \sum_{i=1}^{k} c_i\,\langle v, v_i \rangle = c\,\|v\|^2.$$

Since $v \neq 0$, $\|v\|^2 > 0$ and $c = 0$. Then $c_1 v_1 + \cdots + c_k v_k = 0$, and the independence of $v_1, \dots, v_k$ gives $c_1 = \cdots = c_k = 0$. If $v = \sum_i \alpha_i v_i$, then $1 \cdot v - \sum_i \alpha_i v_i = 0$ would be a nontrivial relation, so $v \notin \operatorname{span}(v_1, \dots, v_k)$. □

Remark. The assumption $v \neq 0$ is needed: the zero vector is orthogonal to every vector, but no set containing it is independent ($1 \cdot 0 = 0$ is a nontrivial relation).

Week: 5 (September 21–25, 2026)

Lecture Date: Thursday, September 24, 2026

Lecture Topic: Deutsch's Algorithm, Phase Flip, Walsh–Hadamard Transform, Deutsch–Jozsa Algorithm

In this lecture, we see the first quantum algorithms that beat every classical algorithm in the query model: Deutsch's algorithm and, using the *phase flip* form of a query and the n-qubit Hadamard (Walsh–Hadamard) transform, the Deutsch–Jozsa algorithm.

1 Deutsch's Algorithm

Problem 2 ▶ Deutsch's Problem

Given a function $f\colon \{0,1\} \to \{0,1\}$ as a black box, determine whether f is *constant* ($f(0) = f(1)$) or *balanced* ($f(0) \neq f(1)$).

Classically we need 2 queries: after one query, say $f(0)$, both answers are still possible. Quantumly, Deutsch's algorithm does it with 1 query.

1.1 The Quantum Query U_f

A quantum circuit is reversible, but f need not be. We therefore convert f into a reversible *quantum version* U_f, a controlled-f gate like the CNOT gate of Definition 11 (Figure 6):

$$U_f\colon |a\rangle|b\rangle \longmapsto |a\rangle|b \oplus f(a)\rangle.$$

Applying U_f twice gives back $|a\rangle|b\rangle$, so U_f is reversible. One application of U_f counts as one query to f.

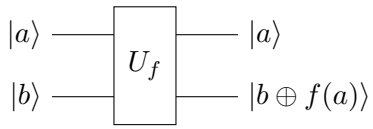


Figure 6: The quantum query U_f.

1.2 The Circuit

In Deutsch's algorithm (Figure 7) the qubits start in $|0\rangle$ and $|1\rangle$, go through a Hadamard gate each, then through U_f, then through a Hadamard gate each, and are measured.

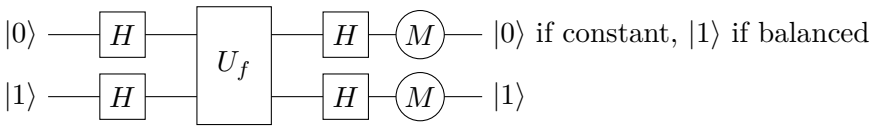


Figure 7: Deutsch's algorithm.

By Equation (3), for $a \in \{0,1\}$,

$$H|a\rangle = \frac{1}{\sqrt{2}}(|0\rangle + (-1)^a|1\rangle) = \frac{1}{\sqrt{2}} \sum_{b\in\{0,1\}} (-1)^{ab}|b\rangle. \tag{6}$$

Exercise 15. Compute $H\,\frac{1}{\sqrt{2}}(|0\rangle + (-1)^a|1\rangle)$ for $a \in \{0,1\}$.

Solution. By linearity and Equation (6) for $H|0\rangle$ and $H|1\rangle$,

$$H\,\tfrac{1}{\sqrt{2}}(|0\rangle + (-1)^a|1\rangle) = \tfrac{1}{2}(|0\rangle + |1\rangle) + \tfrac{(-1)^a}{2}(|0\rangle - |1\rangle) = \tfrac{1+(-1)^a}{2}|0\rangle + \tfrac{1-(-1)^a}{2}|1\rangle.$$

For $a = 0$ this is $|0\rangle$ and for $a = 1$ it is $|1\rangle$, so the result is $|a\rangle$. This agrees with $H^2 = I$ (Lemma 27). □

Exercise 16. In the circuit of Figure 7 the second qubit is measured as $|1\rangle$. What does the measurement of the first qubit give?

The answer is $|0\rangle$ if f is constant and $|1\rangle$ if f is balanced:

Theorem 34 ▶ Deutsch's Algorithm

The circuit of Figure 7 makes one query to f, and measuring its first qubit gives $f(0) \oplus f(1)$ with probability 1. Hence the outcome is 0 if f is constant and 1 if f is balanced.

Proof. After the first two Hadamard gates the state is (by Equation (4)) $|\psi_1\rangle = H|0\rangle \otimes H|1\rangle = \frac{1}{2}(|0\rangle + |1\rangle)(|0\rangle - |1\rangle)$. For $c \in \{0,1\}$ we have $|0 \oplus c\rangle - |1 \oplus c\rangle = (-1)^c(|0\rangle - |1\rangle)$ (for $c = 1$ the two kets are swapped), so U_f maps $|a\rangle(|0\rangle - |1\rangle)$ to $(-1)^{f(a)}|a\rangle(|0\rangle - |1\rangle)$ and, by linearity,

$$|\psi_2\rangle = U_f|\psi_1\rangle = \frac{1}{\sqrt{2}}\Big((-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle\Big) \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle).$$

Since $(-1)^{f(1)} = (-1)^{f(0)}(-1)^{f(0)\oplus f(1)}$, the first qubit is $(-1)^{f(0)}\,\frac{1}{\sqrt{2}}\Big(|0\rangle + (-1)^{f(0)\oplus f(1)}|1\rangle\Big)$, i.e., up to the sign $(-1)^{f(0)}$, the state of Exercise 15 with $a = f(0) \oplus f(1)$; the second qubit is $H|1\rangle$. So the last two Hadamard gates give $|\psi_3\rangle = (-1)^{f(0)}|f(0) \oplus f(1)\rangle|1\rangle$. The global phase $(-1)^{f(0)}$ does not affect the measurement: the first qubit gives $f(0) \oplus f(1)$ with probability 1, and the second always gives 1. □

2 Phase Flip and the n-Qubit Hadamard Transform

If the second (target) qubit of U_f is in the state $\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$, then, by the computation in the proof of Theorem 34, for every input x (also for $x \in \{0,1\}^n$, where $U_f\colon |x\rangle|b\rangle \mapsto |x\rangle|b \oplus f(x)\rangle$),

$$U_f\,|x\rangle\tfrac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = (-1)^{f(x)}\,|x\rangle\tfrac{1}{\sqrt{2}}(|0\rangle - |1\rangle).$$

We drop the target qubit and write the query in the *phase flip* form

$$U_f\colon |x\rangle \longmapsto (-1)^{f(x)}|x\rangle.$$

By linearity, U_f maps $\sum_x \alpha_x|x\rangle$ to $\sum_x \alpha_x(-1)^{f(x)}|x\rangle$: one query flips the sign of every $|x\rangle$ with $f(x) = 1$.

2.1 The n-Qubit Hadamard Transform

To generalize Deutsch's algorithm to n bits we apply the Hadamard gate to each of n qubits. Start with $n = 2$ and let $|x\rangle = |x_1x_2\rangle = |x_1\rangle|x_2\rangle$. Using Equation (4) and then Equation (6) on

each qubit,

$$(H \otimes H)|x\rangle = H|x_1\rangle \otimes H|x_2\rangle = \left(\frac{1}{\sqrt{2}} \sum_{y_1 \in \{0,1\}} (-1)^{x_1 y_1} |y_1\rangle\right)\left(\frac{1}{\sqrt{2}} \sum_{y_2 \in \{0,1\}} (-1)^{x_2 y_2} |y_2\rangle\right)$$
$$= \frac{1}{2} \sum_{y \in \{0,1\}^2} (-1)^{x_1 y_1 + x_2 y_2} |y\rangle = \frac{1}{2} \sum_{y \in \{0,1\}^2} (-1)^{x \cdot y} |y\rangle,$$

where $x \cdot y = x_1 y_1 \oplus x_2 y_2$, since only the parity of the exponent matters. For example, $(H \otimes H)|01\rangle = \frac{1}{2}(|00\rangle - |01\rangle + |10\rangle - |11\rangle)$. In general, for $x, y \in \{0,1\}^n$, the inner product of the bit strings modulo 2 is

$$x \cdot y = x_1 y_1 \oplus x_2 y_2 \oplus \cdots \oplus x_n y_n.$$

Lemma 35 ▶ Walsh–Hadamard Transform

The n-qubit Hadamard transform $H^{\otimes n} = H \otimes H \otimes \cdots \otimes H$ (the *Walsh–Hadamard transform*) acts on a basis state $|x\rangle = |x_1 x_2 \cdots x_n\rangle$ as

$$H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle.$$

Proof. By Equation (4) applied repeatedly, $H^{\otimes n}|x\rangle = H|x_1\rangle \otimes \cdots \otimes H|x_n\rangle$, and by Equation (6) the ith factor is $\frac{1}{\sqrt{2}} \sum_{y_i \in \{0,1\}} (-1)^{x_i y_i} |y_i\rangle$. Multiplying out the n sums,

$$H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y \in \{0,1\}^n} (-1)^{x_1 y_1 + \cdots + x_n y_n} |y\rangle,$$

and (-1) to the power $x_1 y_1 + \cdots + x_n y_n$ only depends on its parity $x \cdot y$. □

For $x = 0^n = 00\cdots0$ we have $x \cdot y = 0$ for every y, so $H^{\otimes n}|00\cdots0\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle$ is the uniform superposition of all 2^n bit strings.

3 Deutsch–Jozsa Algorithm

Problem 3 ▶ Deutsch–Jozsa Problem

Given $f\colon \{0,1\}^n \to \{0,1\}$ as a black box, with the *promise* that f is either constant or balanced (i.e., $f(x) = 0$ for exactly half of the 2^n inputs), determine which is the case.

This is Deutsch's problem on n bits. It is a *promise problem*: we are told in advance that f is one of the two kinds. Let $N = 2^n$ be the number of inputs.

3.1 Classical Query Complexity

Deterministic: in the worst case $2^{n-1} + 1 = N/2 + 1$ queries are needed, since after $N/2$ equal values f may still be either constant or balanced. The same holds for a randomized algorithm that has to succeed with probability 1.

Randomized (Monte Carlo): query f on α independent uniformly random inputs and output "balanced" if two answers differ, "constant" otherwise. If f is constant, we are always right. If f is balanced, every answer is 0 or 1 with probability $1/2$ each, so all α answers agree, and we

fail, with probability $2 \cdot (1/2)^\alpha = (1/2)^{\alpha-1}$. A constant number of queries gives a constant error probability, and $\alpha = \mathcal{O}(\log N)$ queries give error probability $1/N$.

Quantum: the Deutsch–Jozsa algorithm needs 1 query (and n measurements) and makes no error.

3.2 The Algorithm

The circuit (Figure 8) is Deutsch's circuit with n input qubits: n qubits in $|0\rangle$ and one qubit in $|1\rangle$ go through Hadamard gates and then U_f (one query); then all the qubits go through Hadamard gates again and are measured.

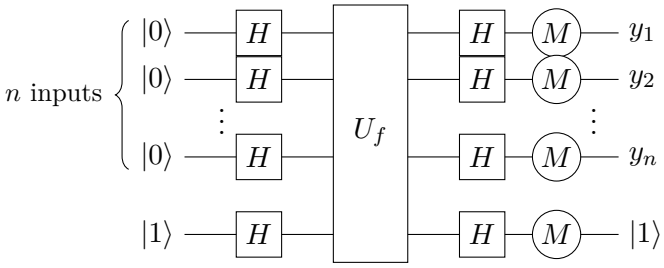


Figure 8: The Deutsch–Jozsa algorithm. The last qubit always gives 1.

Exercise 17. The state of the n input qubits before the measurement is $\frac{1}{2^n}\sum_y \sum_x (-1)^{f(x)+x\cdot y}|y\rangle$. What do the measurements give if f is constant, and what if f is balanced?

Theorem 36 ▶ Deutsch–Jozsa Algorithm

In the circuit of Figure 8, measuring the n input qubits gives $y = 0^n$ with probability 1 if f is constant, and with probability 0 if f is balanced. Hence, if any of these n measurements gives 1, then f is balanced; otherwise f is constant.

Proof. We use the phase flip form of U_f: the last qubit is $\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = H|1\rangle$ before and after the query, so its second Hadamard gate gives $|1\rangle$, and we drop it. The first layer of Hadamard gates maps the input register $|00\cdots0\rangle$ to $\frac{1}{\sqrt{2^n}}\sum_x |x\rangle$, and the query flips the sign of every $|x\rangle$ with $f(x) = 1$. After the second layer of Hadamard gates, by Lemma 35, the state is

$$\frac{1}{\sqrt{2^n}} \sum_{x\in\{0,1\}^n} (-1)^{f(x)} H^{\otimes n}|x\rangle = \frac{1}{2^n} \sum_{y\in\{0,1\}^n} \sum_{x\in\{0,1\}^n} (-1)^{f(x)+x\cdot y}|y\rangle.$$

Since $x \cdot 0^n = 0$ for every x, the amplitude of $|0^n\rangle$ is $\frac{1}{2^n}\sum_x (-1)^{f(x)}$. If f is constant, all 2^n terms are equal and the amplitude is ± 1; the amplitudes squared sum to 1, so all others are 0 and the measurement gives 0^n with probability 1. If f is balanced, half the terms are $+1$ and half are -1, so the amplitude is 0: the measurement gives 0^n with probability 0, i.e., at least one of the n measurements gives 1. □

Week: 6 (September 28 – October 2, 2026)

Lecture Date: Tuesday, September 29, 2026

Lecture Topic: Walsh–Hadamard Transform Recap, Bernstein–Vazirani Algorithm, Simon's Algorithm

In this lecture, we see two more problems that a quantum algorithm solves with far fewer queries than any classical algorithm: the *Bernstein–Vazirani* problem and *Simon's problem*, the first exponential quantum speed-up.

Recap: quantum queries and the Walsh–Hadamard transform. Both algorithms use the query $U_f\colon |x\rangle|b\rangle \mapsto |x\rangle|b\oplus f(x)\rangle$ (Figure 6; one application is one query) and the *Walsh–Hadamard transform* (Lemma 35) $H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}}\sum_{y\in\{0,1\}^n}(-1)^{x\cdot y}|y\rangle$, where $x\cdot y = x_1y_1\oplus\cdots\oplus x_ny_n$.

Lemma 37 ▶ Sum of Signs

For every $z\in\{0,1\}^n$,

$$\sum_{x\in\{0,1\}^n}(-1)^{z\cdot x} = \begin{cases} 2^n & \text{if } z = 0^n, \\ 0 & \text{if } z\neq 0^n. \end{cases}$$

Proof. If $z = 0^n$, all 2^n terms are 1. If $z\neq 0^n$, pick i with $z_i = 1$ and let e_i be the string with a single 1 in position i. The map $x\mapsto x\oplus e_i$ is its own inverse and has no fixed point, so it splits $\{0,1\}^n$ into 2^{n-1} disjoint pairs $\{x, x\oplus e_i\}$. The inner product is linear modulo 2, so $z\cdot(x\oplus e_i) = z\cdot x\oplus z_i = z\cdot x\oplus 1$, and every pair contributes $(-1)^{z\cdot x}+(-1)^{z\cdot x\oplus 1} = 0$. □

In the Deutsch–Jozsa circuit (Figure 8, Theorem 36) the query acts as the phase flip $|k\rangle\mapsto(-1)^{f(k)}|k\rangle$, and the control register after the second layer of Hadamard gates is in the state

$$\sum_{y\in\{0,1\}^n}\alpha_y|y\rangle, \qquad \alpha_y = \frac{1}{2^n}\sum_{k\in\{0,1\}^n}(-1)^{f(k)+k\cdot y}. \tag{7}$$

1 Bernstein–Vazirani Algorithm

Problem 4 ▶ Bernstein–Vazirani Problem

Given $f\colon\{0,1\}^n\to\{0,1\}$ as a black box, with the *promise* that

$$f(x) = s\cdot x = s_1x_1\oplus s_2x_2\oplus\cdots\oplus s_nx_n$$

for some unknown n-bit string s, find s.

This is again a promise problem.

1.1 The Algorithm

The quantum algorithm is the Deutsch–Jozsa circuit (Figure 8); it makes one query.

Theorem 38 ▶ Bernstein–Vazirani Algorithm

Run the Deutsch–Jozsa circuit on $f(x) = s\cdot x$. Then measuring the n control qubits gives $y = s$ with probability 1. Hence s is found with a single query.

Proof. By Equation (7) (with the summation index k renamed x), before the measurement the control register is in the state $\sum_y \alpha_y |y\rangle$ with

$$\alpha_y = \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{s \cdot x \oplus y \cdot x} = \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{(s \oplus y) \cdot x},$$

since only the parity of the exponent matters and the inner product is bilinear modulo 2. By Lemma 37 with $z = s \oplus y$, $\alpha_y = 1$ if $y = s$ and $\alpha_y = 0$ if $y \neq s$. So the state is $|s\rangle$, and the measurement gives s with probability 1. □

Query complexity versus running time. The query model only counts the calls to U_f; the running time in the *quantum Turing machine* (or quantum circuit) model counts all the gates, and the two are not always the same.

2 Simon's Algorithm

Problem 5 ▶ Simon's Problem

Given $f\colon \{0,1\}^n \to \{0,1\}^n$ as a black box, with the *promise* that there is a string $s \in \{0,1\}^n$ such that for all x, y,

$$f(x) = f(y) \iff x = y \text{ or } x = y \oplus s \qquad \text{(equivalently } s = x \oplus y\text{)},$$

find s.

If $s \neq 0^n$, then f is 2-*to*-1: the inputs come in pairs $\{x, x \oplus s\}$, the two inputs of a pair map to the same output, and different pairs map to different outputs. If $s = 0^n$, then f is 1-*to*-1. So the problem asks whether f is 1-to-1 or 2-to-1 and, in the second case, for the period s of f. We write $N = 2^n$.

Query complexity. Classically, Simon's problem needs $\Omega\left(\sqrt{N}\right) = \Omega\left(\sqrt{2^n}\right)$ queries, even for randomized algorithms (next lecture). Simon's quantum algorithm uses $\mathcal{O}(n)$ queries; its classical part is a Gaussian elimination on an $n \times n$ system of linear equations over $\{0,1\}$, which takes $\mathcal{O}(n^3)$ time. This is an exponential speed-up.

2.1 The Circuit

The circuit (Figure 9) has two registers of n qubits each, all starting in $|0\rangle$. The control register goes through Hadamard gates, then both registers go through U_f (one query), then the control register goes through Hadamard gates again and is measured, giving a string $y \in \{0,1\}^n$. Since f has n output bits, we use U_f in its original form $|x\rangle|b\rangle \mapsto |x\rangle|b \oplus f(x)\rangle$ and not the phase flip form.

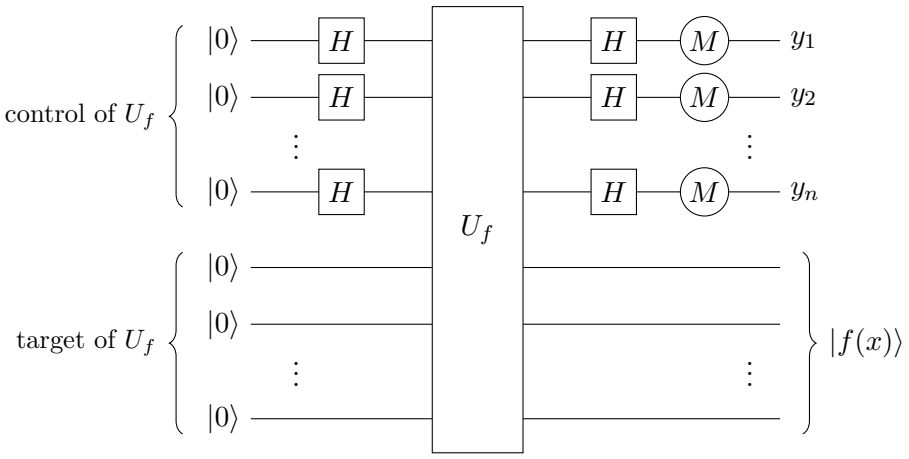


Figure 9: Simon's circuit. The target register is not measured.

2.2 Analysis of One Run

Theorem 39 ▶ One Run of Simon's Circuit

Let y be the string measured by the circuit of Figure 9.

1. If f is 2-to-1 with period $s \neq 0^n$, then y is uniformly distributed over the 2^{n-1} strings y with $s \cdot y = 0$: each of them has probability $1/2^{n-1}$, and every y with $s \cdot y = 1$ has probability 0.
2. If f is 1-to-1, then y is uniformly distributed over all of $\{0,1\}^n$: each string has probability $1/2^n$.

Proof. By Lemma 35, the first layer of Hadamard gates gives $\frac{1}{\sqrt{2^n}} \sum_x |x\rangle|0^n\rangle$, the query U_f turns this into $\frac{1}{\sqrt{2^n}} \sum_x |x\rangle|f(x)\rangle$, and the second layer of Hadamard gates on the control register (Lemma 35 applied to every $|x\rangle$) gives

$$\frac{1}{2^n} \sum_{y \in \{0,1\}^n} \sum_{x \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle|f(x)\rangle. \tag{8}$$

The amplitude $\alpha_{y,z}$ of a basis state $|y\rangle|z\rangle$ collects the x with $f(x) = z$, and measuring only the first register gives y with probability $\sum_z |\alpha_{y,z}|^2$ (the probabilities of all basis states $|y\rangle|z\rangle$ with this y add up).

Case 2: f is 1-to-1. The 2^n strings $f(x)$ are all different, so every basis state $|y\rangle|f(x)\rangle$ appears exactly once in Equation (8), with amplitude $\frac{1}{2^n}(-1)^{x \cdot y}$. Measuring the first register gives y with probability $\sum_x \left|\frac{1}{2^n}(-1)^{x \cdot y}\right|^2 = 2^n \cdot \frac{1}{2^{2n}} = \frac{1}{2^n}$.

Case 1: f is 2-to-1 with period $s \neq 0^n$. The range R of f has $|R| = 2^{n-1}$ elements. For every $z \in R$ let x_z be one of its two preimages; the other is $x_z \oplus s$. Since $(x_z \oplus s) \cdot y = x_z \cdot y \oplus s \cdot y$, the amplitude of $|y\rangle|z\rangle$ in Equation (8) is

$$\frac{1}{2^n}\left((-1)^{x_z \cdot y} + (-1)^{(x_z \oplus s) \cdot y}\right) = \frac{1}{2^n}(-1)^{x_z \cdot y}(1 + (-1)^{s \cdot y})$$

for $z \in R$ (and 0 for $z \notin R$), and measuring the first register gives y with probability

$$\sum_{z \in R} \left|\frac{1}{2^n}(1 + (-1)^{s \cdot y})\right|^2 = 2^{n-1} \cdot \frac{(1 + (-1)^{s \cdot y})^2}{2^{2n}} = \begin{cases} 1/2^{n-1} & \text{if } s \cdot y = 0, \\ 0 & \text{if } s \cdot y = 1. \end{cases}$$

There are 2^{n-1} strings y with $s \cdot y = 0$: the pairing $y \mapsto y \oplus e_i$ of the proof of Lemma 37, with $s_i = 1$, matches every y with $s \cdot y = 0$ to a y' with $s \cdot y' = 1$. □

2.3 From the Measurements to s

Each run of the circuit gives a random y with $s \cdot y = 0$, i.e., one linear equation $s_1y_1 \oplus s_2y_2 \oplus \cdots \oplus s_ny_n = 0$ for the unknown bits $s_1, \ldots, s_n$. Running the circuit $n-1$ times gives a homogeneous system

$$y_1 \cdot s = 0, \quad y_2 \cdot s = 0, \quad \ldots, \quad y_{n-1} \cdot s = 0$$

of linear equations over the field $\{0,1\}$ (addition is $\oplus$). If $y_1, \ldots, y_{n-1}$ are *linearly independent* over $\{0,1\}$, the system has rank $n-1$, so its solution space has dimension $n-(n-1)=1$ and consists of 0^n and one nonzero string s'. Gaussian elimination over $\{0,1\}$ tests the independence and computes s' in $\mathcal{O}(n^3)$ time. The true s satisfies every equation (so if $s \neq 0^n$ at most $n-1$ of them can be independent, and $n-1$ independent ones are enough); hence if the y_i are independent and f is 2-to-1, then $s' = s$.

Algorithm 1 Simon's algorithm

run the circuit of Figure 9 $n-1$ times; let $y_1, \ldots, y_{n-1}$ be the measured strings
if $y_1, \ldots, y_{n-1}$ are linearly dependent over $\{0,1\}$: ▷ *Gaussian elimination*
    return FAIL
let s' be the unique nonzero solution of $y_1 \cdot s' = \cdots = y_{n-1} \cdot s' = 0$
if $f(s') = f(0^n)$: ▷ *two classical queries*
    return s' ▷ *f is 2-to-1 with period s'*
else:
    return 0^n ▷ *f is 1-to-1*

Theorem 40 ▶ Simon's Algorithm

Algorithm 1 makes $n+1$ queries ($n-1$ quantum, 2 classical) and $\mathcal{O}(n^3)$ other operations. It never outputs a wrong answer, and it outputs the correct s with probability at least $1/4$.

Proof. Never wrong. If the algorithm does not fail, $y_1, \ldots, y_{n-1}$ are independent and s' is the unique nonzero solution of the system. If f is 2-to-1 with period s, then s is a nonzero solution, so $s' = s$, and the test $f(s') = f(0^n) = f(0^n \oplus s)$ succeeds: the output $s' = s$ is correct. If f is 1-to-1, then $f(s') \neq f(0^n)$ because $s' \neq 0^n$, and the output $0^n = s$ is correct.

Success probability. The algorithm fails only if the y_i are dependent. The $n-1$ runs are independent (the registers are re-initialized to $|0^n\rangle|0^n\rangle$ for every run). By Theorem 39, the y_i are independent uniform samples from the 2^{n-1} strings with $s \cdot y = 0$ if f is 2-to-1, and from all of $\{0,1\}^n$ if f is 1-to-1. By Theorem 48 (next lecture) they are linearly independent with probability

$$\left(1-\frac{1}{2}\right)\left(1-\frac{1}{2^2}\right)\left(1-\frac{1}{2^3}\right)\cdots\left(1-\frac{1}{2^{n-1}}\right) \geqslant \frac{1}{4}$$

in the first case and at least $1/2$ in the second. □

2.4 A Deterministic Classical Algorithm

Classically, $\mathcal{O}\left(\sqrt{2^n}\right)$ queries suffice even without randomness.

Exercise 18. Find a deterministic classical algorithm for Simon's problem that makes $\mathcal{O}\left(\sqrt{2^n}\right)$ queries.

Solution. We query a set S such that every nonzero string is the XOR of two elements of S; a collision in S then reveals s. Let $m = \lceil n/2 \rceil$ and write every n-bit string as $x = (u, v)$ with $u \in \{0,1\}^m$ its first m bits and $v \in \{0,1\}^{n-m}$ its last $n-m$ bits. Let

$$S_1 = \{(u, 0^{n-m}) \mid u \in \{0,1\}^m\}, \qquad S_2 = \left\{(0^m, v) \mid v \in \{0,1\}^{n-m}\right\}, \qquad S = S_1 \cup S_2.$$

The algorithm. Query f on every $x \in S$. If there are $x \neq x'$ in S with $f(x) = f(x')$, output $s = x \oplus x'$; otherwise output $s = 0^n$.

Queries. Since $n - m \leqslant m \leqslant (n+1)/2$, the algorithm makes $|S| \leqslant 2^m + 2^{n-m} \leqslant 2 \cdot 2^m \leqslant 2 \cdot 2^{(n+1)/2} = 2\sqrt{2} \cdot \sqrt{2^n} = \mathcal{O}\left(\sqrt{2^n}\right)$ queries, and it is deterministic.

Claim: every $s \neq 0^n$ is the XOR of two different elements of S. Write $s = (u, v)$. Then $(u, 0^{n-m}) \in S_1$, $(0^m, v) \in S_2$ and $(u, 0^{n-m}) \oplus (0^m, v) = s$. The two elements are different, since otherwise $u = 0^m$ and $v = 0^{n-m}$, i.e., $s = 0^n$.

Correctness. If f is 2-to-1 with period $s \neq 0^n$, by the claim there are $x \neq x'$ in S with $x' = x \oplus s$, so $f(x) = f(x')$ and a collision is found. Any collision $f(x) = f(x')$ with $x \neq x'$ satisfies $x' = x \oplus s$ by the promise, so the output $x \oplus x' = s$ is correct whichever collision is found. If f is 1-to-1 there is no collision, and the output $0^n = s$ is correct. □

Week: 6 (September 28 – October 2, 2026)

Lecture Date: Thursday, October 1, 2026

Lecture Topic: Classical Query Complexity of Simon's Problem, Yao's Minmax Principle, Linear Independence in Simon's Algorithm

---

In this lecture, we show that the *classical* query complexity of Simon's problem is $\Theta\left(\sqrt{N}\right)$, where $N = 2^n$ (the lower bound by Yao's minmax principle), and we complete the analysis of Simon's quantum algorithm: the measured strings are linearly independent with probability at least $1/4$.

1 Simon's Problem, Classically

Recall Simon's problem: $f\colon \{0,1\}^n \to \{0,1\}^n$ is given as a black box, and we are promised that f is either 1-to-1, or 2-to-1 with a period $s \neq 0^n$, i.e.,

$$f(x) = f(y) \iff x = y \quad \text{or} \quad x \oplus y = s \text{ (i.e., } y = x \oplus s).$$

We have to find s (with $s = 0^n$ meaning that f is 1-to-1). Let $N = 2^n$. A classical algorithm learns f only through queries $f(q_1), f(q_2), \ldots$, and we may assume that it never queries the same input twice. A *collision* is a pair of queries $q_i \neq q_j$ with $f(q_i) = f(q_j)$. By the promise a collision gives $s = q_i \oplus q_j$.

1.1 Deterministic Algorithms

Exercise 18 gives a deterministic algorithm with $\mathcal{O}\left(\sqrt{N}\right)$ queries. This is optimal by the lower bound for randomized algorithms proved below (Corollary 45).

Algorithm 2 Randomized classical algorithm for Simon's problem

choose k distinct inputs $q_1, \dots, q_k$ uniformly at random (without replacement) and query them
if $f(q_i) = f(q_j)$ for some $i \neq j$:
    return $q_i \oplus q_j$
else:
    return 0^n

1.2 A Randomized Algorithm

As in the birthday paradox (lecture of September 3), we query random inputs and wait for a collision.

Theorem 41 ▶ Randomized Upper Bound

If f is 1-to-1, Algorithm 2 is always correct. If f is 2-to-1 with period s, it is wrong with probability at most $e^{-k(k-1)/(2N)}$. In particular, with $k \geqslant \sqrt{2N \ln N} + 1$ queries (an integer at most N once $n \geqslant 3$) the algorithm is correct with probability at least $1 - 1/N$, and with $k \geqslant \sqrt{2N} + 1$, i.e., $\mathcal{O}\left(\sqrt{N}\right)$, queries with probability at least $1 - 1/e$.

Proof. If f is 1-to-1 there is no collision, and the output 0^n is correct. Let f be 2-to-1 with period $s \neq 0^n$. A collision gives $q_i \oplus q_j = s$ by the promise, so the algorithm is wrong only if no collision occurs. Call $q_j \oplus s$ the *mirror* of q_j; a collision occurs at step i exactly when q_i is the mirror of an earlier query.

Suppose that no collision occurred among $q_1, \dots, q_{i-1}$. Their $i-1$ mirrors are distinct and none of them is among $q_1, \dots, q_{i-1}$ (that would be a collision), so all of them are among the $N - (i-1)$ inputs not yet queried, and q_i is uniform among those. Hence the probability of a collision at step i, given none before, is $(i-1)/(N-i+1) \geqslant (i-1)/N$. Multiplying over the k steps and using $1 - x \leqslant e^{-x}$,

$$\begin{aligned}\mathbb{P}(\text{no collision among } q_1, \dots, q_k) &\leqslant \prod_{i=1}^{k} \left(1 - \frac{i-1}{N}\right) \leqslant e^{-\sum_{i=1}^{k} (i-1)/N} \\ &= e^{-k(k-1)/(2N)} \leqslant e^{-(k-1)^2/(2N)}.\end{aligned}$$

With $k - 1 \geqslant \sqrt{2N \ln N}$ this is at most $e^{-\ln N} = 1/N$, and with $k - 1 \geqslant \sqrt{2N}$ it is at most e^{-1}. □

1.3 Lower Bound for Randomized Algorithms via Yao's Principle

Theorem 42 ▶ Randomized Lower Bound

Let $n \geqslant 2$. Every randomized classical algorithm that solves Simon's problem with success probability at least $2/3$ on every input makes more than $\sqrt{N}/2$ queries on some input. Hence every such algorithm needs $\Omega\left(\sqrt{N}\right) = \Omega\left(\sqrt{2^n}\right)$ queries.

Lemma 43 ▶ Yao's Minmax Principle, Easy Direction

Let μ be a probability distribution on the inputs. If every deterministic algorithm that makes at most k queries errs with probability greater than δ on an input drawn from μ, then every randomized algorithm that makes at most k queries errs with probability greater than δ on some input.

Proof. A randomized algorithm R is a deterministic algorithm A_r with at most k queries for every fixed random string r. Let $\mathrm{err}(A, x) \in \{0, 1\}$ indicate that A is wrong on input x. Then

$$\max_x \mathbb{P}_r(\mathrm{err}(A_r, x) = 1) \geqslant \mathbb{E}_{x\sim\mu}[\mathbb{P}_r(\mathrm{err}(A_r, x) = 1)] = \mathbb{E}_r[\mathbb{P}_{x\sim\mu}(\mathrm{err}(A_r, x) = 1)] > \delta,$$

since a maximum is at least an average, the two expectations can be exchanged, and every inner probability on the right is greater than δ by assumption. □

The input distribution. Let $\mathcal{D}$ be the distribution of random 2-to-1 functions: choose s uniformly among the $N - 1$ nonzero strings, which splits $\{0, 1\}^n$ into $N/2$ pairs $\{x, x \oplus s\}$, and assign to the pairs distinct values, uniformly at random among all such assignments. On such an input an algorithm has to output s.

Principle of deferred decisions. We generate the input lazily. Choose s and keep it hidden. When the algorithm queries q_i: if $q_i = q_j \oplus s$ for an earlier query q_j (a *collision*), answer $f(q_j)$; otherwise answer a uniformly random value among those not used so far ($N - (i - 1)$ of them as long as no collision has occurred). This produces $\mathcal{D}$, because a uniformly random injective assignment can be built one value at a time, each uniform among the unused ones.

Lemma 44 ▶ Few Queries Rarely Collide

Let $n \geqslant 2$ and let A be a deterministic algorithm that makes $k \leqslant \sqrt{N}/2$ queries, all different, on an input drawn from $\mathcal{D}$.

1. The probability that two of the queries form a pair $\{x, x \oplus s\}$ (a collision) is at most
$$\sum_{i=1}^{k} \frac{i-1}{N - 1 - \binom{i-1}{2}} \leqslant \frac{k(k-1)}{N}.$$
2. Given that there is no collision among the k queries, and given the k answers, s is uniformly distributed over a set of at least $N/2$ strings.

Proof. Let Q_i be the event that there is no collision among the first i queries, and C_i the event that the ith query collides with an earlier one, i.e., $q_i \oplus s \in \{q_1, \ldots, q_{i-1}\}$.

What the algorithm knows about s. Fix $i \leqslant k + 1$ and condition on Q_{i-1} and on the (distinct) answers $f(q_1), \ldots, f(q_{i-1})$. Since A is deterministic, the queries $q_1, \ldots, q_{i-1}$ are the same for every s that leads to these answers. By the lazy generation, this sequence of answers has probability $\prod_{j=1}^{i-1} 1/(N - j + 1)$, the same for every s not of the form $q_j \oplus q_l$ with $j < l \leqslant i - 1$, and probability 0 for an s of this form (q_j and q_l would have collided). By Bayes' rule, given the answers, s is uniform over the nonzero strings that are not among the at most $\binom{i-1}{2}$ strings $q_j \oplus q_l$, a set of at least $N - 1 - \binom{i-1}{2} \geqslant N - 1 - k^2/2 \geqslant N - 1 - N/8 \geqslant N/2$ candidates (as $i - 1 \leqslant k \leqslant \sqrt{N}/2$ and $N \geqslant 4$). For $i = k + 1$ this is part 2.

Part 1. If C_i is the first collision then Q_{i-1} holds, so by the union bound $\mathbb{P}(\text{collision}) \leqslant \sum_{i=1}^{k} \mathbb{P}(C_i \cap Q_{i-1})$. Given Q_{i-1} and the answers, q_i is determined, and C_i occurs exactly when

s is one of the $i-1$ strings $q_i \oplus q_1, \ldots, q_i \oplus q_{i-1}$. Averaging over the answers,

$$\mathbb{P}(C_i \cap Q_{i-1}) \leqslant \frac{i-1}{N-1-\binom{i-1}{2}} \leqslant \frac{2(i-1)}{N}, \qquad \text{so} \qquad \mathbb{P}(\text{collision}) \leqslant \sum_{i=1}^{k} \frac{2(i-1)}{N} = \frac{k(k-1)}{N}.$$

□

Proof of Theorem 42. Let A be a deterministic algorithm with at most $\sqrt{N}/2$ queries. We may assume that it never repeats a query and makes the same number $k \leqslant \sqrt{N}/2$ of queries on every input (extra queries whose answers are ignored do not change the output). Run it on an input drawn from $\mathcal{D}$; it is correct only if its output is s. If there is no collision, then given the answers the output is one fixed string, while s is uniform over at least $N/2$ strings by part 2 of Lemma 44, so A is correct with probability at most $2/N$. With part 1,

$$\mathbb{P}(A \text{ correct}) \leqslant \mathbb{P}(\text{collision}) + \frac{2}{N} \leqslant \frac{k(k-1)+2}{N}.$$

If $k \geqslant 2$, then $k(k-1)+2 \leqslant k^2 \leqslant N/4$ and the bound is at most $1/4$. If $k \leqslant 1$, there is no collision and the bound is $2/N \leqslant 1/2$. So A errs with probability at least $1/2 > 1/3$ on $\mathcal{D}$, and by Lemma 43 (with $\mu = \mathcal{D}$, $\delta = 1/3$) every randomized algorithm with at most $\sqrt{N}/2$ queries errs with probability greater than $1/3$ on some input. Hence an algorithm with success probability at least $2/3$ on every input makes more than $\sqrt{N}/2$ queries on some input. □

Corollary 45 ▶ Classical Query Complexity of Simon's Problem

The classical query complexity of Simon's problem is $\Theta\left(\sqrt{N}\right) = \Theta\left(\sqrt{2^n}\right)$, both for deterministic algorithms and for randomized algorithms with constant success probability. Simon's quantum algorithm, with $\mathcal{O}(n)$ queries, is therefore exponentially faster than every classical algorithm.

Proof. A deterministic algorithm that is always correct is a randomized algorithm with success probability $1 \geqslant 2/3$, so Theorem 42 gives the lower bound for both kinds. The upper bounds are Exercise 18 and Theorem 41 (the bound $e^{-(k-1)^2/(2N)}$ in its proof is at most $1/3$ for $k \geqslant \sqrt{2N \ln 3} + 1$). □

The same bound holds if the algorithm only has to tell the two cases apart.

Corollary 46 ▶ Decision Version

Let $n \geqslant 2$. Every randomized classical algorithm that decides, with success probability at least $2/3$ on every input, whether f is 1-to-1 or 2-to-1 makes more than $\sqrt{N}/2$ queries on some input.

Proof. Let $\mathcal{P}$ be the uniform distribution on 1-to-1 functions (the table of values is a uniformly random permutation of $\{0,1\}^n$), and let μ draw f from $\mathcal{P}$ or from $\mathcal{D}$ with probability $1/2$ each. Let A be a deterministic algorithm with $k \leqslant \sqrt{N}/2$ queries, all different, as in the proof of Theorem 42. Its answer is a function of the sequence $T = (f(q_1), \ldots, f(q_k))$ of answers.

Let $t = (a_1, \ldots, a_k)$ have distinct entries. Under $\mathcal{P}$ every answer is uniform among the unused values, so $\mathbb{P}_{\mathcal{P}}(T = t) = \prod_{i=1}^{k} 1/(N-i+1)$. Under $\mathcal{D}$, let p_i be the probability of a collision at step i, given no collision before and the answers $a_1, \ldots, a_{i-1}$. By the proof of Lemma 44, $p_i \leqslant 2(i-1)/N$, and without a collision the ith answer is uniform among the $N-i+1$ unused

values. Hence, by Lemma 47 below,

$$\mathbb{P}_{\mathcal{D}}(T=t) \geqslant \prod_{i=1}^{k} \frac{1-p_i}{N-i+1} \geqslant \left(1 - \sum_{i=1}^{k} \frac{2(i-1)}{N}\right) \mathbb{P}_{\mathcal{P}}(T=t)$$
$$= \left(1 - \frac{k(k-1)}{N}\right) \mathbb{P}_{\mathcal{P}}(T=t) \geqslant \frac{3}{4}\,\mathbb{P}_{\mathcal{P}}(T=t),$$

because $k(k-1) \leqslant k^2 \leqslant N/4$.

Let $\mathcal{T}$ be the set of sequences on which A answers "1-to-1", and let $q = \mathbb{P}_{\mathcal{P}}(T \in \mathcal{T})$; sequences with a repeated entry have probability 0 under $\mathcal{P}$. Summing over $t \in \mathcal{T}$ gives $\mathbb{P}_{\mathcal{D}}(T \in \mathcal{T}) \geqslant \frac{3}{4}q$, so A is correct on an input drawn from μ with probability at most

$$\frac{1}{2}q + \frac{1}{2}\left(1 - \frac{3}{4}q\right) = \frac{1}{2} + \frac{q}{8} \leqslant \frac{5}{8} < \frac{2}{3}.$$

So A errs with probability at least $3/8 > 1/3$ on μ, and the claim follows from Lemma 43 as before. □

Lemma 47

For $a_1, \ldots, a_m \in [0,1]$, $\prod_{i=1}^{m}(1-a_i) \geqslant 1 - \sum_{i=1}^{m} a_i$.

Proof. By induction on m, with equality for $m = 1$: multiplying the hypothesis for $m-1$ by $1 - a_m \geqslant 0$,

$$\prod_{i=1}^{m}(1-a_i) \geqslant \left(1 - \sum_{i=1}^{m-1} a_i\right)(1-a_m) = 1 - \sum_{i=1}^{m} a_i + a_m \sum_{i=1}^{m-1} a_i \geqslant 1 - \sum_{i=1}^{m} a_i.$$ □

2 Simon's Algorithm Revisited

In Simon's circuit (Figure 9, Theorem 39) the state just before the measurement is

$$\sum_{y \in \{0,1\}^n} \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{x \cdot y} |f(x)\rangle |y\rangle$$

(target register written first), so the probability of measuring y in the control register is the sum of the squared amplitudes of all basis states $|z\rangle|y\rangle$ with this y, i.e., the squared norm of the vector that multiplies $|y\rangle$:

$$\mathbb{P}(y) = \left\| \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{x \cdot y} |f(x)\rangle \right\|_2^2.$$

2.1 The 1-to-1 Case

Exercise 19. Suppose that f is 1-to-1. What is the probability of measuring a given string y in Simon's circuit, and do these probabilities add up to 1?

Solution. If f is 1-to-1, the 2^n vectors $|f(x)\rangle$ are distinct basis states, hence orthonormal, so the vector multiplying $|y\rangle$ has 2^n orthogonal components, each of absolute value $1/2^n$. Its squared norm (not its norm, which is $1/\sqrt{2^n}$) is

$$\mathbb{P}(y) = \sum_{x \in \{0,1\}^n} \left(\frac{1}{2^n}\right)^2 = 2^n \cdot \frac{1}{2^{2n}} = \frac{1}{2^n}.$$

So the measured y is uniformly distributed over $\{0,1\}^n$, and the probabilities add up to $2^n \cdot 1/2^n = 1$. □

2.2 The 2-to-1 Case

For f 2-to-1 with period $s \neq 0^n$, collecting the two preimages x_z and $x_z \oplus s$ of every z in the range R of f turns the vector into $\frac{1}{2^n}\sum_{z\in R}(-1)^{x_z\cdot y}(1+(-1)^{s\cdot y})|z\rangle$, whose squared norm is $2^{n-1}\cdot 4/2^{2n} = 1/2^{n-1}$ if $s\cdot y = 0$ and 0 if $s\cdot y = 1$ (Theorem 39, case 1). So we only ever measure strings y with $s\cdot y = 0$, each of the 2^{n-1} such strings equally likely, and the total is $2^{n-1}\cdot 1/2^{n-1} = 1$.

2.3 Solving for s

Running the circuit $n-1$ times gives strings $y_1,\dots,y_{n-1}$, and in the 2-to-1 case the system of linear equations $s\cdot y_1 = 0,\dots,s\cdot y_{n-1} = 0$ over $\{0,1\}$. If the y_i are *linearly independent*, the $(n-1)\times n$ coefficient matrix has rank $n-1$, and Gaussian elimination gives a one-dimensional solution space $\{0^n, s'\}$. We test $f(0^n) = f(s')$ with two classical queries. If the test is true, then f is 2-to-1 and $s = s'$ by the promise (as $s' \neq 0^n$). If it is false, then f is 1-to-1, since in the 2-to-1 case s itself solves the system, so $s' = s$ and the test would be true. This is Algorithm 1.

Exercise 20. Show that the probability that the y's are linearly independent is at least $1/4$.

Theorem 48 ▶ Linear Independence of Random Vectors

Let V be a subspace of $\{0,1\}^n$ of dimension $d \geqslant 1$, so $|V| = 2^d$, and let $y_1, y_2, \dots$ be independent uniformly random elements of V.

1. $y_1,\dots,y_d$ are linearly independent with probability
$$\prod_{i=1}^{d}\left(1-\frac{1}{2^i}\right) = \left(1-\frac{1}{2}\right)\left(1-\frac{1}{2^2}\right)\left(1-\frac{1}{2^3}\right)\cdots\left(1-\frac{1}{2^d}\right) \geqslant \frac{1}{4}.$$
2. $y_1,\dots,y_{d-1}$ are linearly independent with probability $\prod_{i=2}^{d}(1-1/2^i) \geqslant 1/2$.

In particular, in the 2-to-1 case ($V = \{y \mid s\cdot y = 0\}$, $d = n-1$) the strings $y_1,\dots,y_{n-1}$ of Simon's algorithm are linearly independent with probability at least $1/4$, and in the 1-to-1 case ($V = \{0,1\}^n$, $d = n$) with probability at least $1/2$.

Proof. Suppose that $y_1,\dots,y_j$ are linearly independent, $0 \leqslant j < d$. Their span consists of the 2^j vectors $\sum_{i\leqslant j} c_i y_i$ with $c_i \in \{0,1\}$, all different by independence, and it lies inside the subspace V. The vector y_{j+1} is uniform over the 2^d elements of V, independent of $y_1,\dots,y_j$, and $y_1,\dots,y_{j+1}$ are linearly independent exactly when y_{j+1} is not in this span, which has probability $1 - 2^j/2^d = 1 - 2^{-(d-j)}$. Multiplying these conditional probabilities for $j = 0,\dots,m-1$, the first m vectors are linearly independent with probability

$$\prod_{j=0}^{m-1}\left(1-\frac{1}{2^{d-j}}\right) = \prod_{i=d-m+1}^{d}\left(1-\frac{1}{2^i}\right),$$

which gives the two products for $m = d$ and $m = d-1$. By Lemma 47 and the geometric series $\sum_{i\geqslant 2} 2^{-i} = 1/2$,

$$\prod_{i=2}^{d}\left(1-\frac{1}{2^i}\right) \geqslant 1 - \sum_{i=2}^{d}\frac{1}{2^i} \geqslant \frac{1}{2}, \qquad \prod_{i=1}^{d}\left(1-\frac{1}{2^i}\right) = \frac{1}{2}\prod_{i=2}^{d}\left(1-\frac{1}{2^i}\right) \geqslant \frac{1}{4}.$$

For Simon's algorithm: in the 2-to-1 case each y_i is uniform over the 2^{n-1} strings with $s \cdot y = 0$ (computed above), the solution set of one nontrivial homogeneous linear equation and hence a subspace of dimension $d = n - 1$, and we take $n - 1 = d$ samples; in the 1-to-1 case each y_i is uniform over $\{0,1\}^n$ (Exercise 19), $d = n$, and we take $d - 1$ samples. □

Week: 7 (October 5–9, 2026)

Lecture Date: Tuesday, October 6, 2026

Lecture Topic: Grover's Search Algorithm, Phase Flip, Diffusion Operator, Rotation Analysis

In this lecture, we study *Grover's search algorithm*, which finds a solution of an unstructured search problem over $N = 2^n$ inputs with $\mathcal{O}\left(\sqrt{N}\right)$ quantum queries, whereas every classical algorithm needs $\Omega(N)$ queries. Every Grover iteration is a rotation by the angle 2θ in a two-dimensional plane, which gives the number of iterations.

Problem 6 ▶ Unstructured Search

Given $f\colon \{0,1\}^n \to \{0,1\}$ as a black box, find an x with $f(x) = 1$.

Think of f as a Boolean formula and of x as an assignment; in *unique satisfiability* we are promised that there is exactly one x with $f(x) = 1$. Let

$$a = |\{x \mid f(x) = 1\}|, \qquad b = |\{x \mid f(x) \neq 1\}| = N - a$$

be the numbers of solutions and of non-solutions, where $N = 2^n$. The search is *unstructured*: we know nothing about f except what the queries tell us. For $a = 1$ every classical algorithm, deterministic or randomized, needs $\Omega(N)$ queries (lectures of September 1 and 3). Grover's algorithm needs $\mathcal{O}\left(\sqrt{N}\right)$ queries, and $\mathcal{O}\left(\sqrt{N/a}\right)$ for a solutions.

1 Grover's Algorithm on Two Qubits

Let $n = 2$, $N = 4$, and suppose that the unique solution is $x^* = 10$. We write a state of the two qubits as its vector of four amplitudes $(a_{00}, a_{01}, a_{10}, a_{11})$. The algorithm has four steps (Figure 10).

1. *Uniform superposition with Hadamard gates:* $H^{\otimes 2}|00\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle) = \left(\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}\right)$.

2. *Phase flip.* The query $U_f\colon |x\rangle|c\rangle \mapsto |x\rangle|c \oplus f(x)\rangle$ with the target qubit in $\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$ is the phase flip of the lecture of September 24, $U_f\colon |x\rangle \mapsto (-1)^{f(x)}|x\rangle$, which flips the sign of the amplitude of every solution. One query gives $\left(\frac{1}{2}, \frac{1}{2}, -\frac{1}{2}, \frac{1}{2}\right)$.

3. *Grover diffusion operator.* Let $\mu = \frac{1}{N}\sum_x a_x$ be the mean of the amplitudes. The diffusion operator D reflects every amplitude about the mean (*inversion about the mean*):

$$D\colon a_x \longmapsto 2\mu - a_x \qquad \text{for every } x.$$

   Here $\mu = \frac{1}{4}$, so the three non-solutions become $2 \cdot \frac{1}{4} - \frac{1}{2} = 0$ and the solution becomes $2 \cdot \frac{1}{4} + \frac{1}{2} = 1$: the state is $(0,0,1,0) = |10\rangle$, and measuring gives $x^* = 10$ with probability 1 after a single query.

4. In general, *repeat steps 2 and 3* k times, where k is about $\sqrt{N}$ (Section 3). For $N = 4$ one iteration is enough.

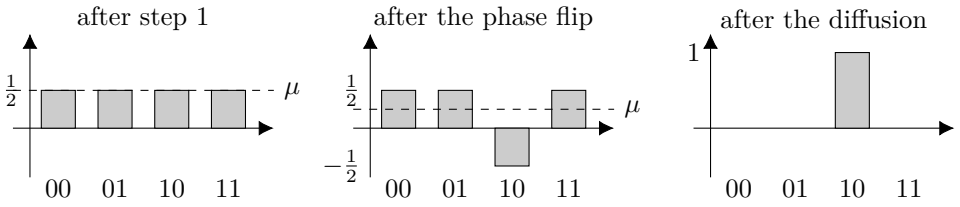


Figure 10: The amplitudes for $N = 4$ and $x^* = 10$ after the Hadamard gates, after the phase flip, and after the inversion about the mean μ.

2 Grover's Algorithm on n Qubits

For n qubits and a unique solution x^* the same steps apply to the N amplitudes. After the Hadamard gates every amplitude is $1/\sqrt{N}$, and the phase flip makes the amplitude of x^* equal to $-1/\sqrt{N}$, so the mean is $\mu = (N-2)/(N\sqrt{N}) \approx 1/\sqrt{N}$ and the diffusion operator maps the amplitude of x^* to $2\mu + 1/\sqrt{N} \approx 3/\sqrt{N}$. As long as this amplitude is small, the mean stays close to $1/\sqrt{N}$ and each iteration adds about $2/\sqrt{N}$ to it, so after about $\sqrt{N}$ iterations it is a constant and a measurement finds x^* with constant probability.

2.1 The Operators Z_f, Z_0 and D

The phase flip Z_f. The phase flip

$$Z_f \colon |x\rangle \longmapsto (-1)^{f(x)}|x\rangle$$

is implemented by one query U_f with an extra qubit in $H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$ (Figure 11): U_f multiplies $|x\rangle$ by $(-1)^{f(x)}$ and leaves the extra qubit unchanged, and a second Hadamard gate returns it to $|1\rangle$.

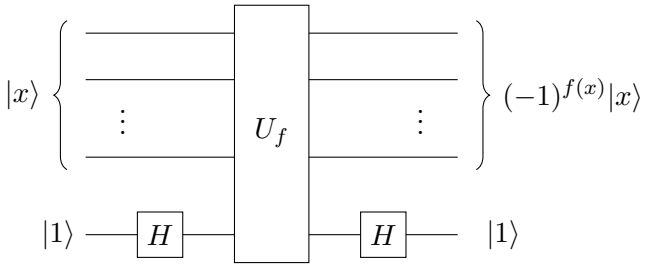


Figure 11: The phase flip Z_f, implemented with one query U_f.

The phase flip Z_0 about 0^n. Let $f_0 \colon \{0,1\}^n \to \{0,1\}$ be the function with $f_0(x) = 1$ only for $x = 0^n$, i.e., $f_0(x) = \bar{x}_1 \wedge \bar{x}_2 \wedge \cdots \wedge \bar{x}_n$ where $\bar{x}_i = \neg x_i$. Its phase flip is

$$Z_0 \colon |x\rangle \longmapsto \begin{cases} -|x\rangle & \text{if } x = 0^n, \\ |x\rangle & \text{otherwise,} \end{cases} \qquad \text{i.e.,} \qquad Z_0 = I - 2|0^n\rangle\langle 0^n|.$$

As a matrix, Z_0 is the $N \times N$ diagonal matrix with -1 in the top left corner and 1 elsewhere on the diagonal. It is implemented like Z_f, with the circuit for f_0 in place of the black box: the reversible gate $|x\rangle|c\rangle \mapsto |x\rangle|c \oplus (\bar{x}_1 \wedge \cdots \wedge \bar{x}_n)\rangle$ (a Toffoli gate with n negated controls) and an extra qubit in $H|1\rangle$. It makes no query to f.

The diffusion operator D. Let $|h\rangle = H^{\otimes n}|0^n\rangle = \frac{1}{\sqrt{N}}\sum_x |x\rangle$ be the uniform superposition. The diffusion operator is

$$D = -H^{\otimes n} Z_0 H^{\otimes n}.$$

Lemma 49 ▶ Inversion About the Mean

$D = 2|h\rangle\langle h| - I$, and for every state $\sum_x a_x|x\rangle$ with mean amplitude $\mu = \frac{1}{N}\sum_x a_x$,

$$D \sum_{x\in\{0,1\}^n} a_x|x\rangle = \sum_{x\in\{0,1\}^n} (2\mu - a_x)|x\rangle.$$

Proof. Since $H^\dagger = H$ and $H^2 = I$, we have $H^{\otimes n}H^{\otimes n} = I$ and $\langle 0^n|H^{\otimes n} = (H^{\otimes n}|0^n\rangle)^\dagger = \langle h|$, so

$$D = -H^{\otimes n}(I - 2|0^n\rangle\langle 0^n|)H^{\otimes n} = -(I - 2|h\rangle\langle h|) = 2|h\rangle\langle h| - I.$$

For $|\psi\rangle = \sum_x a_x|x\rangle$ we have $\langle h|\psi\rangle = \frac{1}{\sqrt{N}}\sum_x a_x = \sqrt{N}\mu$, so

$$D|\psi\rangle = 2\sqrt{N}\mu|h\rangle - |\psi\rangle = 2\mu\sum_x |x\rangle - \sum_x a_x|x\rangle = \sum_x (2\mu - a_x)|x\rangle. \qquad \square$$

So D is the inversion about the mean of step 3, and it makes no query.

The Grover iteration. One iteration of steps 2 and 3 is the operator

$$G = DZ_f = -H^{\otimes n} Z_0 H^{\otimes n} Z_f.$$

Grover's algorithm prepares $|h\rangle = H^{\otimes n}|0^n\rangle$, applies G k times and measures (Figure 12). It makes k queries, one per iteration.

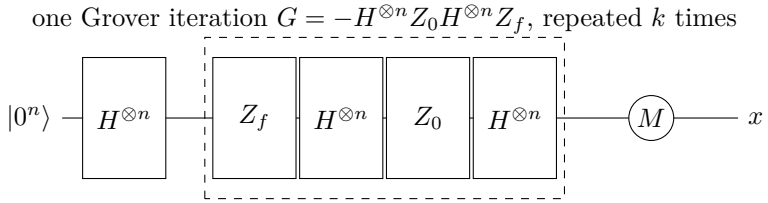


Figure 12: Grover's algorithm on n qubits (the extra qubit used by Z_f and Z_0 is not drawn). Each iteration makes one query.

3 Analysis: A Rotation in Two Dimensions

Let f have a solutions, $1 \leqslant a \leqslant N-1$, and let $b = N - a$. The whole computation takes place in a two-dimensional plane. Let

$$|A\rangle = \frac{1}{\sqrt{a}} \sum_{\substack{x\in\{0,1\}^n \\ f(x)=1}} |x\rangle, \qquad |B\rangle = \frac{1}{\sqrt{b}} \sum_{\substack{x\in\{0,1\}^n \\ f(x)=0}} |x\rangle$$

be the uniform superpositions of the solutions and of the non-solutions. They are unit vectors and orthogonal (no basis state appears in both). The initial state is

$$|h\rangle = \frac{1}{\sqrt{N}} \sum_{x\in\{0,1\}^n} |x\rangle = \frac{1}{\sqrt{N}}\Big(\sqrt{a}|A\rangle + \sqrt{b}|B\rangle\Big) = \sqrt{\frac{a}{N}}|A\rangle + \sqrt{\frac{b}{N}}|B\rangle. \tag{9}$$

Since $\frac{a}{N} + \frac{b}{N} = 1$, there is a unique angle $\theta \in (0, \pi/2)$ with

$$\sin\theta = \sqrt{\frac{a}{N}}, \qquad \cos\theta = \sqrt{\frac{b}{N}}, \qquad \text{so that} \qquad |h\rangle = \sin\theta|A\rangle + \cos\theta|B\rangle.$$

For a unique solution, $\sin\theta = 1/\sqrt{N}$, so $\theta \approx 1/\sqrt{N}$ and $|h\rangle$ is almost $|B\rangle$ (Figure 13).

Lemma 50 ▶ One Grover Iteration

$$G|A\rangle = \left(1 - \frac{2a}{N}\right)|A\rangle - \frac{2\sqrt{ab}}{N}|B\rangle = \cos 2\theta|A\rangle - \sin 2\theta|B\rangle,$$
$$G|B\rangle = \frac{2\sqrt{ab}}{N}|A\rangle - \left(1 - \frac{2b}{N}\right)|B\rangle = \sin 2\theta|A\rangle + \cos 2\theta|B\rangle.$$

Proof. The phase flip Z_f negates every solution and fixes every non-solution, so $Z_f|A\rangle = -|A\rangle$ and $Z_f|B\rangle = |B\rangle$. By Equation (9) and orthonormality, $\langle h|A\rangle = \sqrt{a/N}$ and $\langle h|B\rangle = \sqrt{b/N}$. With $D = 2|h\rangle\langle h| - I$ (Lemma 49), substituting Equation (9) for $|h\rangle$ in the last step of each line,

$$G|A\rangle = -D|A\rangle = |A\rangle - 2|h\rangle\langle h|A\rangle = |A\rangle - 2\sqrt{\frac{a}{N}}|h\rangle = \left(1 - \frac{2a}{N}\right)|A\rangle - \frac{2\sqrt{ab}}{N}|B\rangle,$$
$$G|B\rangle = D|B\rangle = 2|h\rangle\langle h|B\rangle - |B\rangle = 2\sqrt{\frac{b}{N}}|h\rangle - |B\rangle = \frac{2\sqrt{ab}}{N}|A\rangle - \left(1 - \frac{2b}{N}\right)|B\rangle.$$

The trigonometric form follows from $\sin^2\theta = a/N$, $\cos^2\theta = b/N$ and the double angle formulas: $\cos 2\theta = 1 - 2\sin^2\theta = 1 - 2a/N$, $\cos 2\theta = 2\cos^2\theta - 1 = -(1 - 2b/N)$ and $\sin 2\theta = 2\sin\theta\cos\theta = 2\sqrt{ab}/N$. □

Since $a + b = N$, both $1 - 2a/N$ and $2b/N - 1$ equal $(b-a)/N$. We write a state $\beta|B\rangle + \alpha|A\rangle$ of the plane as the column vector $(\beta, \alpha)^T$ in the basis $(|B\rangle, |A\rangle)$. Then G acts as the 2×2 matrix

$$G = \begin{pmatrix} \frac{b-a}{N} & -\frac{2\sqrt{ab}}{N} \\ \frac{2\sqrt{ab}}{N} & \frac{b-a}{N} \end{pmatrix} = \begin{pmatrix} \cos 2\theta & -\sin 2\theta \\ \sin 2\theta & \cos 2\theta \end{pmatrix}, \tag{10}$$

whose columns are the coordinates of $G|B\rangle$ and $G|A\rangle$. The matrix on the right is the rotation matrix $R_\phi = \left(\begin{smallmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{smallmatrix}\right)$ with $\phi = 2\theta$. So one Grover iteration *rotates* the plane by the angle 2θ, from $|B\rangle$ towards $|A\rangle$.

Exercise 21. Compute $G|h\rangle = \sqrt{a/N}\,G|A\rangle + \sqrt{b/N}\,G|B\rangle$, and verify that the matrix of G in Equation (10) is the rotation $R_{2\theta}$.

Solution. *The matrix is a rotation.* By Lemma 50 and $(b-a)/N = 1 - 2a/N = 2b/N - 1$, the two columns of the matrix in Equation (10) are the coordinates of $G|B\rangle = \cos 2\theta|B\rangle + \sin 2\theta|A\rangle$ and $G|A\rangle = -\sin 2\theta|B\rangle + \cos 2\theta|A\rangle$, which are the columns of $R_{2\theta}$ (not R_θ).

The state $G|h\rangle$. By Equation (9), $|h\rangle = \sin\theta|A\rangle + \cos\theta|B\rangle$. By Lemma 50 and the addition formulas for $\sin(\theta + 2\theta)$ and $\cos(\theta + 2\theta)$,

$$\begin{aligned} G|h\rangle &= \sin\theta(\cos 2\theta|A\rangle - \sin 2\theta|B\rangle) + \cos\theta(\sin 2\theta|A\rangle + \cos 2\theta|B\rangle) \\ &= \sin 3\theta|A\rangle + \cos 3\theta|B\rangle. \end{aligned}$$

With $\sin 3\theta = 3\sin\theta - 4\sin^3\theta$, $\cos 3\theta = 4\cos^3\theta - 3\cos\theta$ and $4b/N - 3 = 1 - 4a/N$, this is

$$G|h\rangle = \sqrt{\frac{a}{N}}\left(3 - \frac{4a}{N}\right)|A\rangle + \sqrt{\frac{b}{N}}\left(1 - \frac{4a}{N}\right)|B\rangle.$$

The state starts at the angle θ from $|B\rangle$ and the iteration adds 2θ (Figure 13). □

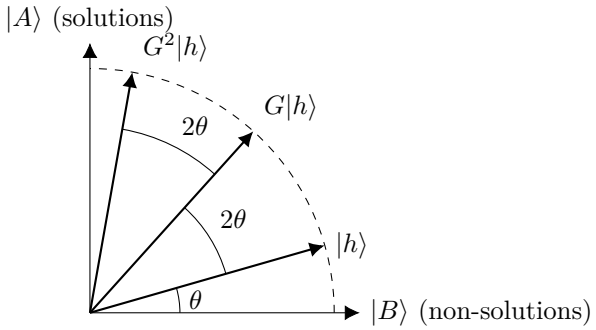


Figure 13: Grover's algorithm in the plane spanned by $|A\rangle$ and $|B\rangle$ for $\theta = 16°$: every iteration rotates the state by 2θ.

The number of iterations is computed in the next lecture (Section 2).

Week: 7 (October 5–9, 2026)

Lecture Date: Thursday, October 8, 2026

Lecture Topic: Grover's Algorithm as Two Reflections, Number of Iterations, Random Number of Iterations, Quantum Amplitude Amplification

---

In this lecture, we finish the analysis of *Grover's search algorithm* from the lecture of October 6: the phase flip Z_f and the diffusion operator D are two *reflections* of a two-dimensional plane, and two reflections make a rotation. We then show that a *random* number of iterations succeeds with constant probability even when the number a of solutions is not known, and we generalize Grover's algorithm to *quantum amplitude amplification*.

Recap. We keep the notation of the last lecture (a, $b = N - a$ with $1 \leqslant a \leqslant N-1$, $|A\rangle$, $|B\rangle$, Z_f, Z_0, D, $|h\rangle$ and the Grover iteration $G = DZ_f = -H^{\otimes n}Z_0H^{\otimes n}Z_f$; Figure 12). The number of iterations, about $\sqrt{N/a}$, depends on a, so a has to be known; Section 3 removes this assumption. By Equation (9), $|h\rangle = \sin\theta|A\rangle + \cos\theta|B\rangle$ with $\theta = \sin^{-1}\left(\sqrt{a/N}\right) \in (0, \pi/2)$. We write a unit vector of the plane spanned by $|A\rangle$ and $|B\rangle$ with real coefficients as $\sin\alpha|A\rangle + \cos\alpha|B\rangle$ and call α its *angle* (measured from $|B\rangle$ towards $|A\rangle$). So $|h\rangle$ has angle θ, and a measurement of the state with angle α gives a solution with probability $\sin^2\alpha$.

1 A Grover Iteration Is Two Reflections

Since $Z_f|A\rangle = -|A\rangle$ and $Z_f|B\rangle = |B\rangle$ (proof of Lemma 50), Z_f keeps the $|B\rangle$ component and negates the $|A\rangle$ component: $Z_f(\sin\alpha|A\rangle + \cos\alpha|B\rangle) = \sin(-\alpha)|A\rangle + \cos(-\alpha)|B\rangle$. It is the reflection of the plane about the line of $|B\rangle$, and it sends the angle α to $-\alpha$.

For the diffusion operator, let $|h^\perp\rangle = \cos\theta|A\rangle - \sin\theta|B\rangle$, a unit vector of the plane with $\langle h|h^\perp\rangle = \sin\theta\cos\theta - \cos\theta\sin\theta = 0$, so $|h\rangle$ and $|h^\perp\rangle$ are a second orthonormal basis of the plane (Figure 14).

Exercise 22. Show that $D = 2|h\rangle\langle h| - I$, and that for every state $|\phi\rangle = c|h\rangle + d|h^\perp\rangle$ of the plane,

$$D|\phi\rangle = c|h\rangle - d|h^\perp\rangle.$$

Solution. The first claim is Lemma 49: $H^{\otimes n}H^{\otimes n} = I$ and $\langle 0^n|H^{\otimes n} = \langle h|$ give

$$D = -H^{\otimes n}(I - 2|0^n\rangle\langle 0^n|)H^{\otimes n} = 2|h\rangle\langle h| - I.$$

Since $\langle h|h\rangle = 1$ and $\langle h|h^\perp\rangle = 0$, we have $\langle h|\phi\rangle = c$, so

$$D|\phi\rangle = 2|h\rangle\langle h|\phi\rangle - |\phi\rangle = 2c|h\rangle - c|h\rangle - d|h^\perp\rangle = c|h\rangle - d|h^\perp\rangle.$$

So D keeps the $|h\rangle$ component and negates the $|h^\perp\rangle$ component: it is the reflection of the plane about the line of $|h\rangle$. □

The sign of G. In class the Grover iteration was also written as $G = (H^{\otimes n}Z_0H^{\otimes n})(-Z_f)$. This is the same operator as $G = DZ_f = (-H^{\otimes n}Z_0H^{\otimes n})Z_f$, because the minus sign can be moved from one factor to the other. In the form DZ_f it is the reflection about $|B\rangle$ followed by the reflection about $|h\rangle$; in the other form $-Z_f$ negates the $|B\rangle$ component and $H^{\otimes n}Z_0H^{\otimes n} = I - 2|h\rangle\langle h|$ negates the $|h\rangle$ component, so it is the reflection about $|A\rangle$ followed by the reflection about $|h^\perp\rangle$. Both pairs of lines make the angle θ.

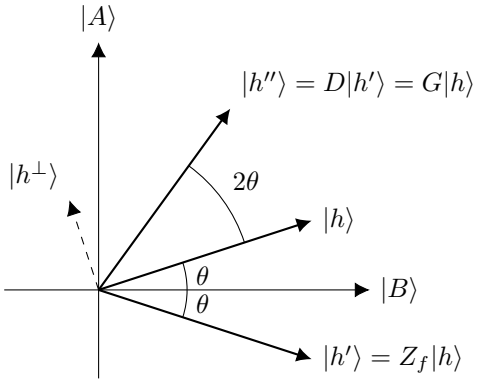


Figure 14: One Grover iteration applied to $|h\rangle$: Z_f reflects about the line of $|B\rangle$, then D reflects about the line of $|h\rangle$.

1.1 Two Reflections Make a Rotation

In Figure 14 the first reflection takes the angle θ to $-\theta$ and the second, about the line at angle θ, takes $-\theta$ to 3θ. We prove this for any two orthonormal vectors and use it again in Section 4.

Lemma 51 ▶ Two Reflections Make a Rotation

Let $|A\rangle$ and $|B\rangle$ be orthonormal vectors, let θ be a real number and $|\psi\rangle = \sin\theta|A\rangle + \cos\theta|B\rangle$. Let S be a linear operator with $S|A\rangle = -|A\rangle$ and $S|B\rangle = |B\rangle$, and let $T = 2|\psi\rangle\langle\psi| - I$. Then for every real number β,

$$TS(\sin\beta|A\rangle + \cos\beta|B\rangle) = \sin(\beta + 2\theta)|A\rangle + \cos(\beta + 2\theta)|B\rangle,$$

and therefore, for every integer $k \geqslant 0$,

$$(TS)^k|\psi\rangle = \sin((2k+1)\theta)|A\rangle + \cos((2k+1)\theta)|B\rangle.$$

Proof. By linearity, $S(\sin\beta|A\rangle + \cos\beta|B\rangle) = -\sin\beta|A\rangle + \cos\beta|B\rangle = \sin\gamma|A\rangle + \cos\gamma|B\rangle =: |\chi\rangle$ with $\gamma = -\beta$. By orthonormality, $\langle\psi|\chi\rangle = \sin\theta\sin\gamma + \cos\theta\cos\gamma = \cos(\gamma-\theta)$, so

$$\begin{aligned} T|\chi\rangle &= 2\cos(\gamma-\theta)|\psi\rangle - |\chi\rangle \\ &= (2\cos(\gamma-\theta)\sin\theta - \sin\gamma)|A\rangle + (2\cos(\gamma-\theta)\cos\theta - \cos\gamma)|B\rangle. \end{aligned}$$

By the product formulas $2\cos u\sin v = \sin(u+v) - \sin(u-v)$ and $2\cos u\cos v = \cos(u+v) + \cos(u-v)$ with $u = \gamma - \theta$ and $v = \theta$, the coefficients are $-\sin(\gamma-2\theta) = \sin(2\theta-\gamma)$ and $\cos(\gamma-2\theta) = \cos(2\theta-\gamma)$. So T sends the angle γ to $2\theta-\gamma$, the mirror image of γ about θ, and TS sends β to $2\theta-(-\beta) = \beta+2\theta$, which is the first claim. As $|\psi\rangle$ has the angle θ and each application of TS adds 2θ, induction on k gives the angle $(2k+1)\theta$ for $(TS)^k|\psi\rangle$. □

For Grover's algorithm we take $S = Z_f$, $|\psi\rangle = |h\rangle$ and $T = D = 2|h\rangle\langle h| - I$ (Exercise 22), so that $TS = DZ_f = G$:

Corollary 52 ▶ The State After k Iterations

Every Grover iteration rotates the plane by 2θ from $|B\rangle$ towards $|A\rangle$, and the state after k iterations is

$$|h^k\rangle = G^k|h\rangle = \sin((2k+1)\theta)|A\rangle + \cos((2k+1)\theta)|B\rangle.$$

A measurement of $|h^k\rangle$ gives a solution with probability $\sin^2((2k+1)\theta)$.

1.2 The Matrix of a Grover Iteration

Let M be the matrix of the Grover iteration in the basis $(|B\rangle, |A\rangle)$, i.e., the matrix of Equation (10), which follows from Lemma 50.

Exercise 23. Show that

$$M = \begin{pmatrix} \sqrt{b/N} & -\sqrt{a/N} \\ \sqrt{a/N} & \sqrt{b/N} \end{pmatrix}^2 = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}^2,$$

i.e., that the Grover iteration is the rotation by θ applied twice.

Solution. Let $R_\theta = \left(\begin{smallmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{smallmatrix}\right)$ with $\cos\theta = \sqrt{b/N}$ and $\sin\theta = \sqrt{a/N}$. Multiplying,

$$R_\theta^2 = \begin{pmatrix} \cos^2\theta - \sin^2\theta & -2\sin\theta\cos\theta \\ 2\sin\theta\cos\theta & \cos^2\theta - \sin^2\theta \end{pmatrix} = \begin{pmatrix} \dfrac{b-a}{N} & -\dfrac{2\sqrt{ab}}{N} \\ \dfrac{2\sqrt{ab}}{N} & \dfrac{b-a}{N} \end{pmatrix} = M,$$

and by the double angle formulas the middle matrix is $R_{2\theta}$. Hence $M^k = R_{2k\theta}$, which is Corollary 52 in matrix form: $|h\rangle$ is the first column $(\cos\theta, \sin\theta)^T$ of R_θ, so $M^k|h\rangle$ is the first column of $R_{(2k+1)\theta}$. □

2 The Number of Iterations

By Corollary 52 the success probability after k iterations is $\sin^2((2k+1)\theta)$, so we want $(2k+1)\theta \approx \pi/2$, i.e., $k \approx \frac{\pi}{4\theta} - \frac{1}{2}$. With $\theta \approx \sin\theta = \sqrt{a/N}$ we take

$$k = \left\lfloor \frac{\pi}{4}\sqrt{\frac{N}{a}} \right\rfloor, \qquad \text{for a unique solution} \quad k = \left\lfloor \frac{\pi}{4}\sqrt{N} \right\rfloor.$$

Exercise 24. The best number of iterations is $k \approx \frac{\pi}{4\theta} - \frac{1}{2}$. But k has to be an integer, and the formulas above replace θ by $\sin\theta = \sqrt{a/N}$. Show that Grover's algorithm with $k = \lfloor \frac{\pi}{4}\sqrt{N/a} \rfloor$ iterations still finds a solution with probability close to 1.

Theorem 53 ▶ Number of Iterations

Let f have a solutions, $1 \leqslant a \leqslant N/2$, and let $\kappa = \frac{\pi}{4}\sqrt{N/a}$. Grover's algorithm with $k = \lfloor \kappa \rfloor$ iterations makes $k \geqslant 1$ queries and outputs a solution with probability at least $1 - 2a/N$. For a unique solution this is $k = \lfloor \frac{\pi}{4}\sqrt{N} \rfloor$ and probability at least $1 - 2/N$.

Proof. Let $t = \frac{\pi}{4\theta} - \frac{1}{2}$, so that $(2t+1)\theta = \pi/2$ and $(2k+1)\theta = \frac{\pi}{2} + 2(k-t)\theta$ for every integer k. By Corollary 52, $\sin(\pi/2 + x) = \cos x$ and $|\sin x| \leqslant |x|$, the success probability is

$$\sin^2((2k+1)\theta) = \cos^2(2(k-t)\theta) = 1 - \sin^2(2(k-t)\theta) \geqslant 1 - 4(k-t)^2\theta^2. \tag{11}$$

Two facts about θ. Since $\sin\theta = \sqrt{a/N} \leqslant 1/\sqrt{2}$, we have $0 < \theta \leqslant \pi/4$, so $\theta < 0.79$ and $\theta^2 < 0.62$. We use $\theta - \theta^3/6 \leqslant \sin\theta \leqslant \theta$ for $\theta \geqslant 0$ (the lower bound holds because $g(x) = \sin x - x + x^3/6$ has $g(0) = 0$ and $g'(x) = \cos x - 1 + x^2/2 \geqslant 0$, as $1 - \cos x = 2\sin^2(x/2) \leqslant x^2/2$). So $\sin\theta \geqslant \theta(1 - \theta^2/6)$ with $1 - \theta^2/6 > 1 - 0.62/6 > 0.896$. Hence, first, as $0.896^2 > 0.8$,

$$\theta^2 \leqslant \frac{\sin^2\theta}{(1-\theta^2/6)^2} < \frac{\sin^2\theta}{0.896^2} < \frac{5}{4}\sin^2\theta = \frac{5a}{4N}. \tag{12}$$

Second, $\kappa = \frac{\pi}{4\sin\theta}$, and

$$0 \leqslant \frac{1}{\sin\theta} - \frac{1}{\theta} \leqslant \frac{1}{\theta(1-\theta^2/6)} - \frac{1}{\theta} = \frac{\theta}{6-\theta^2} < \frac{0.79}{5.38} < 0.15,$$

so, multiplying by $\pi/4 < 0.79$, we get $0 \leqslant \kappa - \pi/4\theta < 0.79 \cdot 0.15 < 1/8$ and therefore $\frac{1}{2} \leqslant \kappa - t < \frac{5}{8}$.

The bound. For $k = \lfloor \kappa \rfloor$ we have $-1 < k - \kappa \leqslant 0$, so $-\frac{1}{2} < k - t = (k - \kappa) + (\kappa - t) < \frac{5}{8}$, i.e., $(k-t)^2 < 25/64$. By Equations (11) and (12) the success probability is at least

$$1 - 4 \cdot \frac{25}{64} \cdot \frac{5a}{4N} = 1 - \frac{125}{64} \cdot \frac{a}{N} \geqslant 1 - \frac{2a}{N}.$$

Moreover $\kappa \geqslant \frac{\pi}{4}\sqrt{2} > 1$ because $N/a \geqslant 2$, so $k \geqslant 1$. □

The bracket can also be read as rounding up; that reading does not give the bound in general: for $a = N/4$ we have $\theta = \pi/6$ and $\kappa = \pi/2 \approx 1.57$, so $\lfloor \kappa \rfloor = 1$ iteration gives the angle $\pi/2$ and success probability 1, while $\lceil \kappa \rceil = 2$ iterations overshoot to $5\pi/6$ and success probability $1/4$.

3 A Random Number of Iterations

The number of iterations $\lfloor \frac{\pi}{4}\sqrt{N/a} \rfloor$ depends on the number a of solutions, which we may not know. So we choose k *at random*.

Exercise 25. Prove that if k is chosen uniformly at random from $\left\{1, \ldots, 2\sqrt{N}\right\}$ and then k Grover iterations are made, then $\mathbb{P}(\text{success}) = \Theta(1)$.

Since $\sqrt{N}$ need not be an integer, the set is $\left\{1, \ldots, \lfloor 2\sqrt{N} \rfloor\right\}$. The algorithm does not use a.

Theorem 54 ▶ Grover's Algorithm with a Random Number of Iterations

Let $N = 2^n$, let f have a solutions, $1 \leqslant a \leqslant N/2$, and let $K = \lfloor 2\sqrt{N} \rfloor$. Choose k uniformly at random from $\{1, \ldots, K\}$, make k Grover iterations on $|h\rangle$ and measure. Then the probability of measuring a solution is

$$\mathbb{P}(\text{success}) = \frac{1}{2} - \frac{\sin(4(K+1)\theta) - \sin 4\theta}{4K \sin 2\theta} \geqslant \frac{1}{2} - \frac{1}{4K \sin 2\theta} \geqslant \frac{3}{8}. \tag{13}$$

So $\mathbb{P}(\text{success}) = \Theta(1)$, and the algorithm makes at most $2\sqrt{N} = \mathcal{O}\left(\sqrt{N}\right)$ queries.

Proof. The average. Given k, the success probability is $\sin^2((2k+1)\theta)$ by Corollary 52, and each value has probability $1/K$. By the law of total probability and $\sin^2 x = \frac{1}{2}(1 - \cos 2x)$,

$$\mathbb{P}(\text{success}) = \frac{1}{K} \sum_{k=1}^{K} \sin^2((2k+1)\theta) = \frac{1}{2} - \frac{1}{2K} \sum_{k=1}^{K} \cos((4k+2)\theta).$$

By the product formula $2 \cos u \sin v = \sin(u+v) - \sin(u-v)$ with $u = (4k+2)\theta$ and $v = 2\theta$, the sum telescopes:

$$2 \sin 2\theta \sum_{k=1}^{K} \cos((4k+2)\theta) = \sum_{k=1}^{K} (\sin(4(k+1)\theta) - \sin(4k\theta)) = \sin(4(K+1)\theta) - \sin 4\theta.$$

As $\sin\theta = \sqrt{a/N} \leqslant 1/\sqrt{2}$, we have $0 < \theta \leqslant \pi/4$ and $\sin 2\theta > 0$, so dividing by $2 \sin 2\theta$ gives the equality in Equation (13).

The lower bound. $\sin(4(K+1)\theta) \leqslant 1$, and $\sin 4\theta \geqslant 0$ because $0 < 4\theta \leqslant \pi$, so the numerator is at most 1, which gives the first inequality in Equation (13). Since $b = N - a \geqslant N/2$,

$$\sin 2\theta = 2 \sin\theta \cos\theta = \frac{2\sqrt{ab}}{N} \geqslant \frac{2\sqrt{a \cdot N/2}}{N} = \sqrt{\frac{2a}{N}}.$$

Next, $\lfloor 2\sqrt{N} \rfloor \geqslant \sqrt{2N}$ for $N = 2^n$: if n is even, $2\sqrt{N}$ is an integer; if n is odd, $\sqrt{2N} = 2^{(n+1)/2}$ is an integer with $\sqrt{2N} \leqslant 2\sqrt{N}$, so it is at most the floor. Therefore $K \sin 2\theta \geqslant \sqrt{2N} \cdot \sqrt{2a/N} = 2\sqrt{a} \geqslant 2$ and $\mathbb{P}(\text{success}) \geqslant \frac{1}{2} - \frac{1}{8} = \frac{3}{8}$. A probability is at most 1, so $\mathbb{P}(\text{success}) = \Theta(1)$. The number of queries is $k \leqslant K \leqslant 2\sqrt{N}$. □

For the set $\left\{1, \ldots, \lfloor \sqrt{N} \rfloor\right\}$ the same proof with $K = \lfloor \sqrt{N} \rfloor \geqslant \sqrt{N/2}$ gives $\mathbb{P}(\text{success}) \geqslant 1/4$.

4 Quantum Amplitude Amplification

Let $\mathcal{A}$ be any quantum algorithm without measurements, i.e., a unitary operator on n qubits, and let $f\colon \{0,1\}^n \to \{0,1\}$ tell us which outcomes x are *good* ($f(x) = 1$) and which are *bad* ($f(x) = 0$). Write $\mathcal{A}|0^n\rangle = \sum_x c_x |x\rangle$, let $p = \sum_{x|f(x)=1} |c_x|^2$ be the probability that a measurement of $\mathcal{A}|0^n\rangle$ gives a good outcome, and assume $0 < p < 1$. With the unit vectors

$$|G\rangle = \frac{1}{\sqrt{p}} \sum_{x|f(x)=1} c_x |x\rangle, \qquad |B\rangle = \frac{1}{\sqrt{1-p}} \sum_{x|f(x)=0} c_x |x\rangle,$$

which are orthogonal because no basis state appears in both, the state that $\mathcal{A}$ produces is $|\phi\rangle = \mathcal{A}|0^n\rangle = \sqrt{p}\,|G\rangle + \sqrt{1-p}\,|B\rangle$. *Classically* we would run $\mathcal{A}$, measure, check the outcome with f, and repeat until the outcome is good; the number of runs is geometric with mean $1/p$. *Amplitude amplification* needs only $\mathcal{O}(1/\sqrt{p})$ runs.

Exercise 26. Show that a good outcome can be found with constant probability using $\mathcal{O}(1/\sqrt{p})$ applications of $\mathcal{A}$ and of its inverse, instead of the $1/p$ repetitions of the classical method.

Theorem 55 ▶ Amplitude Amplification

Let $\mathcal{A}$, f, p, $|G\rangle$, $|B\rangle$ and $|\phi\rangle$ be as above, let $\theta \in (0, \pi/2)$ with $\sin\theta = \sqrt{p}$, and let

$$Q = -\mathcal{A}Z_0\mathcal{A}^{-1}Z_f.$$

Then $Q^k|\phi\rangle = \sin((2k+1)\theta)|G\rangle + \cos((2k+1)\theta)|B\rangle$ for every $k \geqslant 0$. For $k = \lfloor \pi/4\theta \rfloor$, a measurement of $Q^k|\phi\rangle$ gives a good outcome with probability at least $\max(p, 1-p) \geqslant 1/2$, and computing $Q^k\mathcal{A}|0^n\rangle$ takes $2k+1 \leqslant \frac{\pi}{2\sqrt{p}} + 1 = \mathcal{O}(1/\sqrt{p})$ applications of $\mathcal{A}$ or $\mathcal{A}^{-1}$ and k queries to f.

Proof. The two reflections. Z_f negates every good basis state and fixes every bad one, so $Z_f|G\rangle = -|G\rangle$ and $Z_f|B\rangle = |B\rangle$. Since $\mathcal{A}$ is unitary, $\mathcal{A}^{-1} = \mathcal{A}^\dagger$, so $\mathcal{A}I\mathcal{A}^{-1} = I$ and $\langle 0^n|\mathcal{A}^{-1} = (\mathcal{A}|0^n\rangle)^\dagger = \langle\phi|$. Hence

$$-\mathcal{A}Z_0\mathcal{A}^{-1} = -\mathcal{A}(I - 2|0^n\rangle\langle 0^n|)\mathcal{A}^{-1} = -(I - 2|\phi\rangle\langle\phi|) = 2|\phi\rangle\langle\phi| - I.$$

So $Q = TS$ with $S = Z_f$ and $T = 2|\phi\rangle\langle\phi| - I$, and $|\phi\rangle = \sin\theta|G\rangle + \cos\theta|B\rangle$ because $\cos\theta = \sqrt{1-\sin^2\theta} = \sqrt{1-p}$. Lemma 51, applied to the orthonormal vectors $|G\rangle$ and $|B\rangle$ and to $|\psi\rangle = |\phi\rangle$, gives the formula for $Q^k|\phi\rangle$.

The success probability. $|G\rangle$ contains only good basis states and $|B\rangle$ only bad ones, so a measurement of $Q^k|\phi\rangle$ gives a good outcome with probability $\sin^2((2k+1)\theta)$. For $k = \lfloor \pi/4\theta \rfloor$ we have $\pi/4\theta - 1 < k \leqslant \pi/4\theta$, so $\frac{\pi}{2} - \theta < (2k+1)\theta \leqslant \frac{\pi}{2} + \theta$. On the interval $[\pi/2 - \theta, \pi/2 + \theta] \subseteq [0, \pi]$ the sine is at least its value at the endpoints, $\sin(\pi/2 \pm \theta) = \cos\theta$, so the probability is at least $\cos^2\theta = 1 - p$. It is also at least p: if $p \leqslant 1/2$ then $1 - p \geqslant p$, and if $p > 1/2$ then $\theta > \pi/4$, so $k = 0$ and the probability is $\sin^2\theta = p$.

The cost. One application of Q uses $\mathcal{A}$ once, $\mathcal{A}^{-1}$ once and Z_f once (one query to f); Z_0 uses neither. Preparing $|\phi\rangle$ uses $\mathcal{A}$ once more, so the total is $2k+1$ applications, and $k \leqslant \frac{\pi}{4\theta} \leqslant \frac{\pi}{4\sin\theta} = \frac{\pi}{4\sqrt{p}}$ because $\sin\theta \leqslant \theta$. □

Here $|G\rangle$ is the good state, unrelated to the Grover iteration G.